Objektorientierte Systemanalyse und Entwurf

Vorlesung 2: Anforderungsmodellierung

Die Inhalte der Vorlesung wurden primär auf Basis der angegebenen Literatur erstellt. Darüber hinaus sind ausgewählte Teile aus der Vorlesung von Prof. Dr.-Ing. Schmietendorf (HWR Berlin) übernommen worden. Für die Bereitstellung dieses Vorlesungsmaterials möchte ich mich an dieser Stelle noch einmal recht herzlich bedanken.

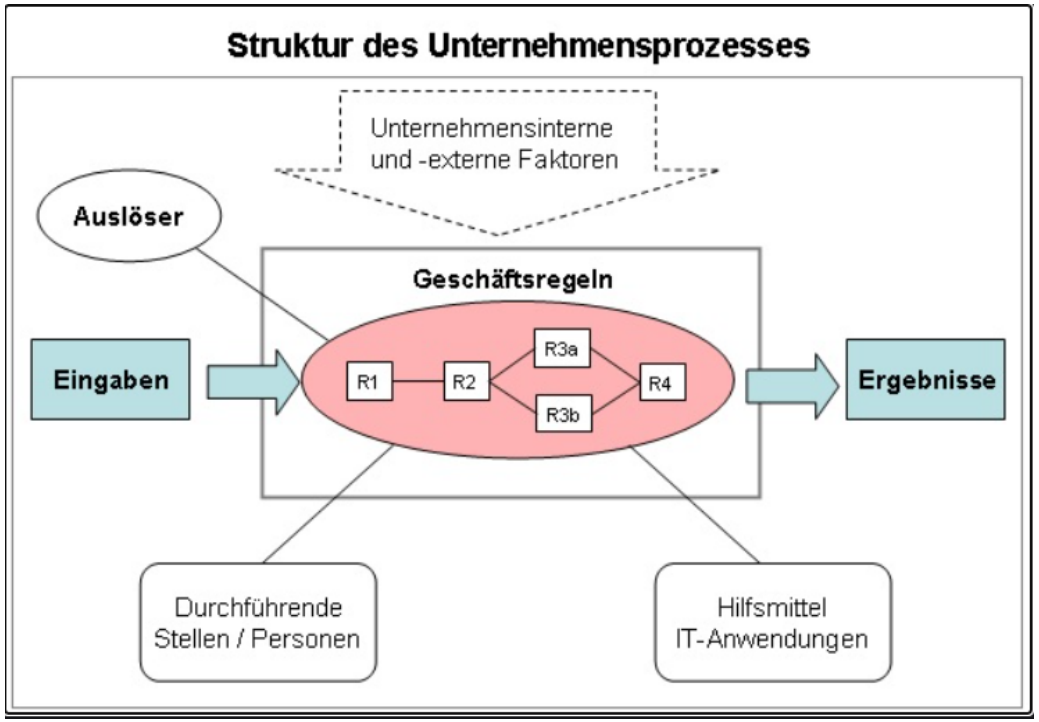
Übersicht zur Vorlesung

- Geschäftsprozesse: Definition & Modellierungsnotationen
  - WKD (Wertschöpfungskettendiagramme)
  - EPK (Ereignisgesteuerte Prozessketten)
- Motivation zur Anforderungsanalyse
- Inhalte & Aufbau eines Pflichtenheft
- Ableitung und Untersetzung von Use Cases
  - Use Cases und Use Case Diagramme
  - Aktivitätsdiagramme
  - Untersetzung von Use Cases durch Aktivitätsdiagramme

Geschäftsprozesse: Definition & Modellierungsnotationen

Geschäftsprozesse: allg. Definition

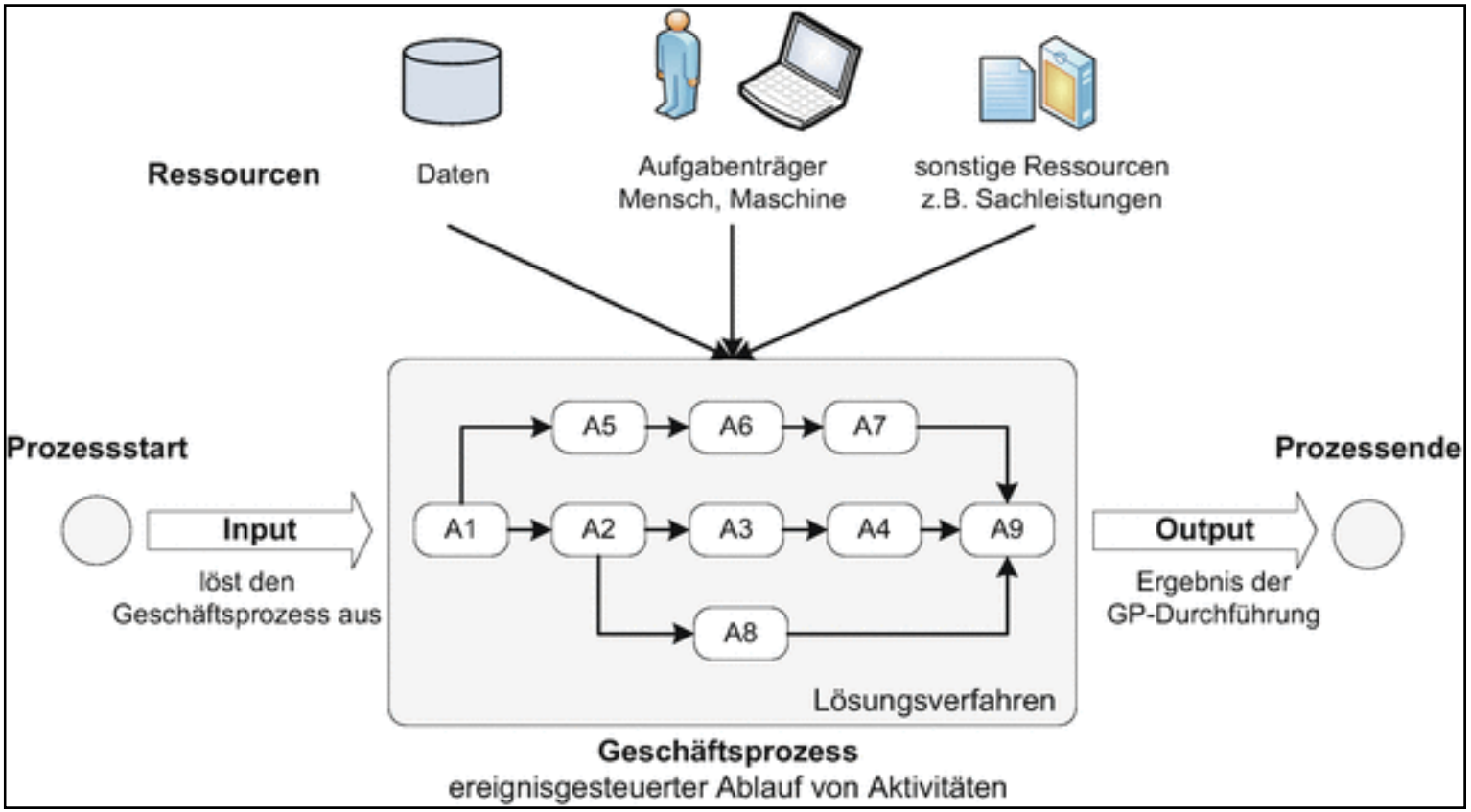
- **Geschäftsprozesse** helfen Unternehmen, Aktivitäten zu strukturieren und in Form von klar definierten Arbeitsabläufen zu organisieren.
- Ein **Geschäftsprozess** hat einen bestimmten **Input** (Eingabe) und einen **Output** (Ergebnis), der einen klar definierten **Mehrwert** liefert.



Quelle: https://de.wikipedia.org/wiki/Gesch%C3%A4ftsprozess, letzter Abruf: 07.10.2025

Geschäftsprozesse: Konkretisierung

Ein **Geschäftsprozess** besteht aus einzelnen Aktivitäten (Teilprozesse). Der Ablauf der Aktivitäten ist ereignisgesteuert.



Quelle: Benker, Th.; Jürck, C.; Wolf, M.: GP-orientierte Systementwicklung, S. 66, 2016

Geschäftsprozessmodellierung: Motivation

„Wenn du das, was du tust, nicht als Prozess beschreiben kannst, dann weißt du nicht, was du tust“ (W. Edwards Demming, Unternehmensberater und Professor an der Columbia Universität (1900-1993))

Wozu dient die Geschäftsprozessmodellierung:

- Dokumentation existierender Geschäftsprozesse (z.B. für Zertifizierungen oder Schulungen)
- Analyse und Optimierung von Geschäftsprozessen
- Neugestaltung von Geschäftsprozessen
- Simulation (z.B. zur Personalbedarfsplanung)
- Prozesskostenrechnung
- Kommunikation zwischen Fachbereich und Softwareentwickler
- modellbasierte Entwicklung

Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

Geschäftsprozessmodellierung: Notationen

Wertschöpfungskettendiagramme (WKD / value chain)

- eingeführt 1985 von Michael E. Porter (Buch: Competitive Advantage - Creating and Sustaining Superior Performance, Free Press 1998, ISBN 978-0-684-84146-5)
- zeigt die Tätigkeiten, die zur Herstellung eines Produkts / einer Dienstleistung im Unternehmen durchgeführt werden

Ereignisgesteuerte Prozessketten (EPK)

- entwickelt 1992 an der Universität des Saarlandes
- in Deutschland (neben BPMN) die verbreitetste Notation zur Geschäftsprozessmodellierung
- u.a. in SAP R/3 verwendet
- graphische, semi-formale Notation

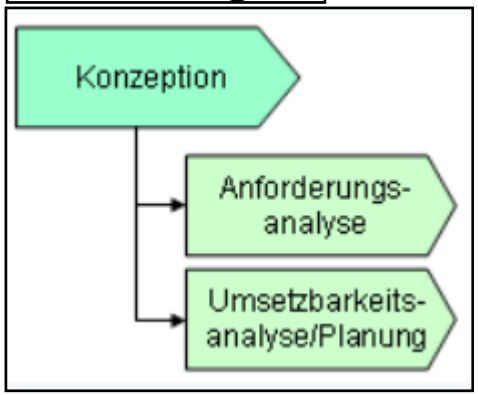
Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

Geschäftsprozesse: WKD-Grundelemente

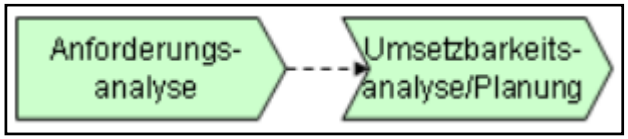
Value Chain



Beziehungen



Aufbaustruktur (Hierarchie)



Ablaufstruktur (Reihenfolge)

Weitere Elemente

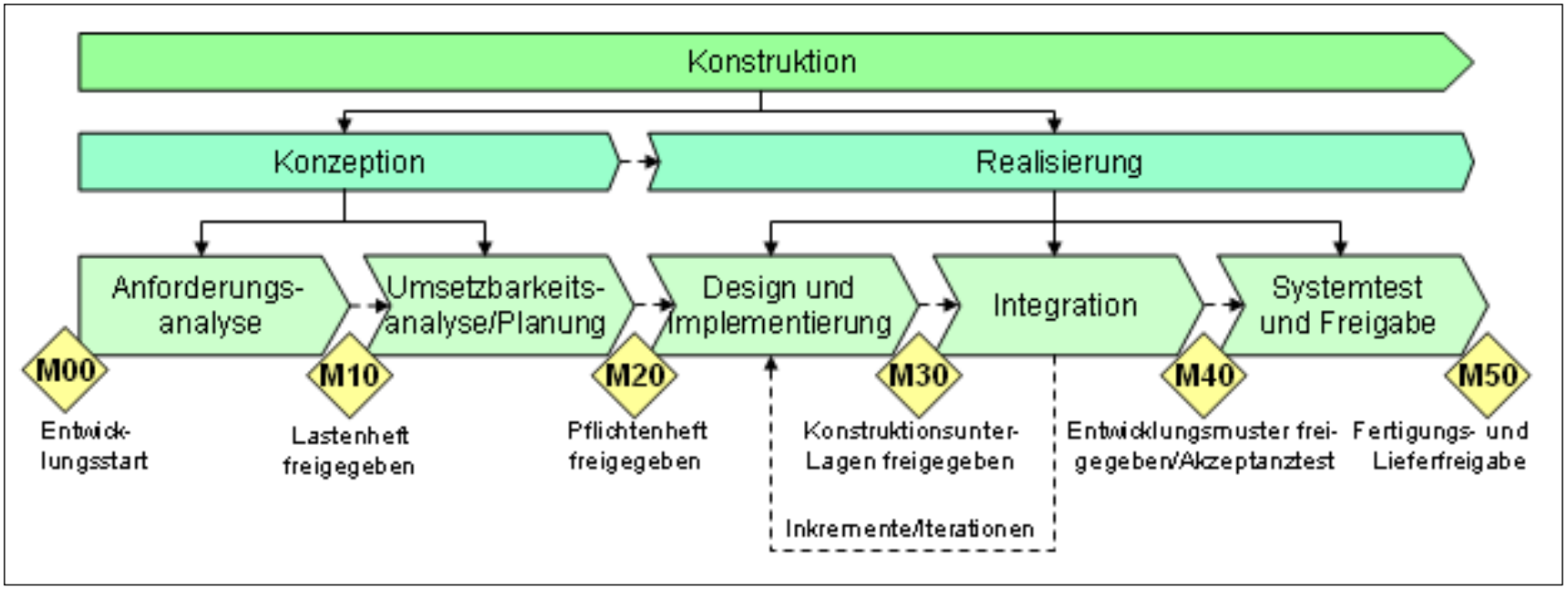
- Organisationseinheiten/Abteilungen
- Datenobjekte
- Anwendungssysteme
- Leistungen
- Ziele
- Messgrößen und
- Risiken

Darstellung von GP

Bei der Darstellung von GP
in WKD werden diese in Teilprozesse
gegliedert, die miteinander in
Beziehung treten.

Quelle: https://de.wikipedia.org/wiki/Wertsch%C3%B6pfungskettendiagramm, letzter Abruf: 07.10.2025

Geschäftsprozesse: WKD-Grundelemente
(Beispiel: Aufbau- und Ablaufstrukturen)



Quelle: https://de.wikipedia.org/wiki/Wertsch%C3%B6pfungskettendiagramm, letzter Abruf: 07.10.2025

Wertschöpfungskette: Prozesse

Managementprozesse (Führungsprozesse, Steuerungsprozesse, Lenkungsprozesse)

- Prozesse der betrieblichen Planung, Steuerung und Kontrolle
- Beispiele: Strategieentwicklung, Erstellen von Zielvorgaben

Kernprozesse (Core Processes, Primäre Prozesse)

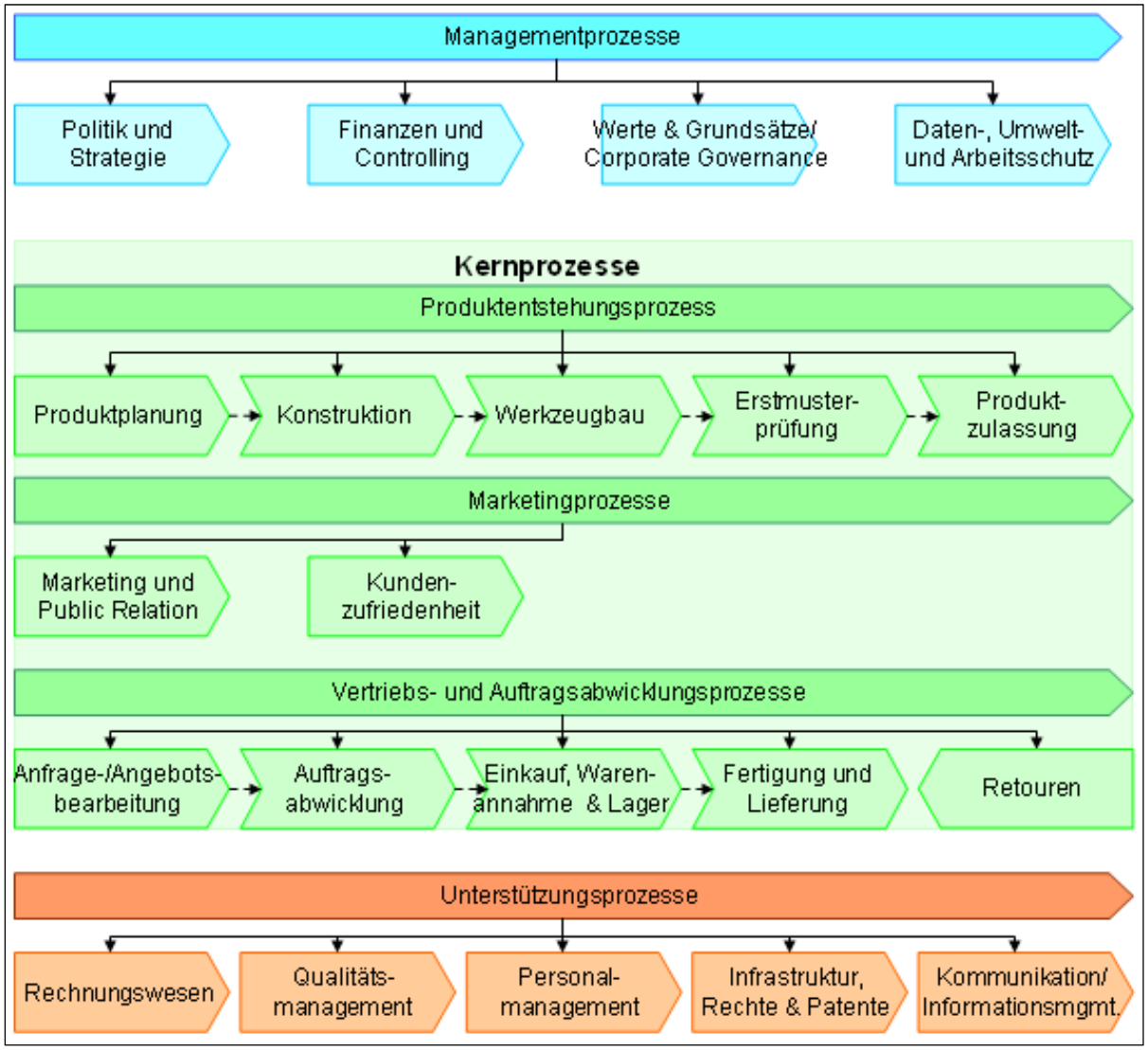
- leisten einen direkten Beitrag zur Entstehung oder dem Vertrieb eines Produktes
- auf externen Kunden ausgerichtetzeigt
- Beispiele: Auftragsbearbeitung, Produktion, Kundendienst, Marketing und Vertrieb

Unterstützende Prozese (Support Processes)

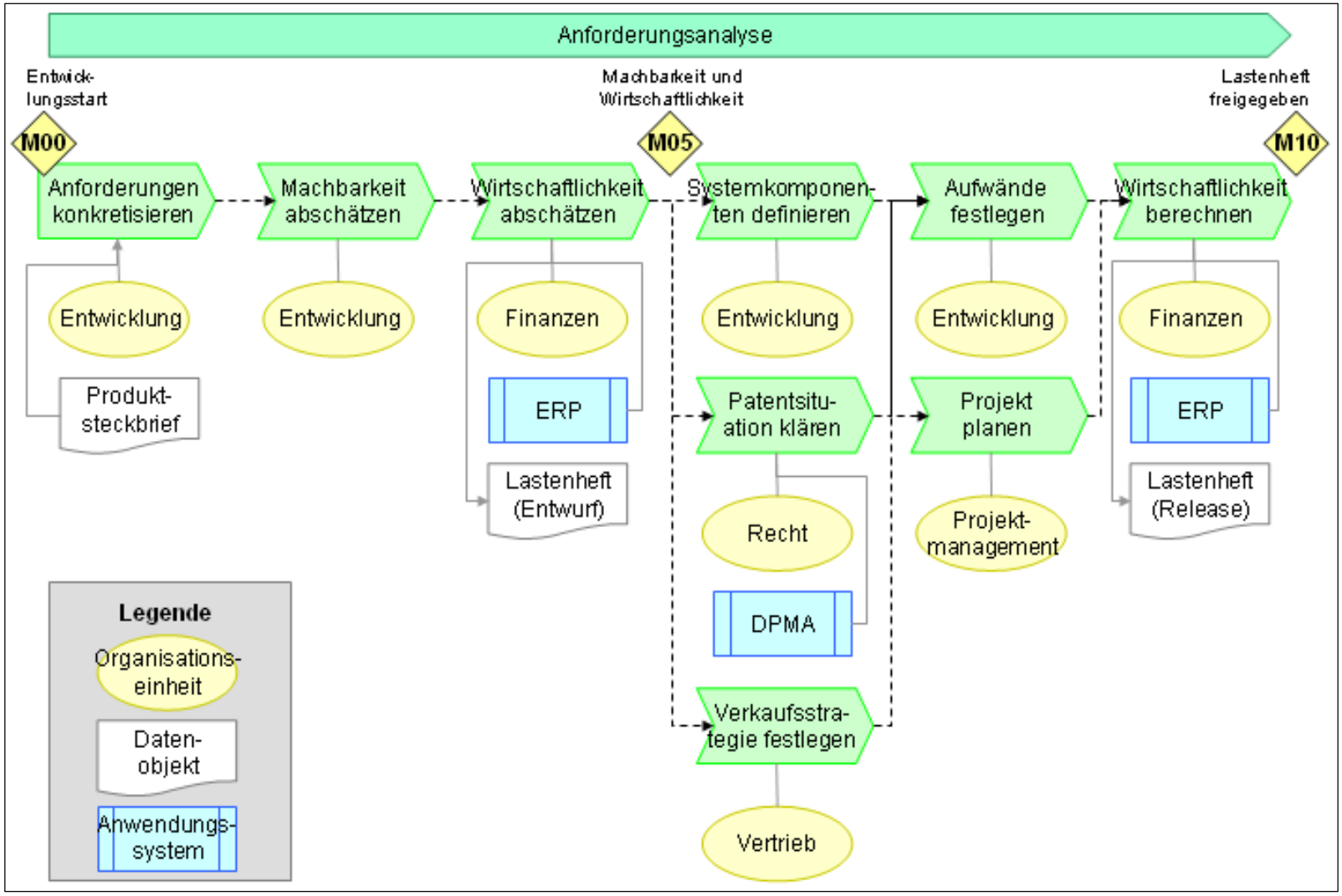
- leisten keinen direkten Beitrag zur Wertschöpfung
- auf interne Kunden ausgerichtet, unterstützen die primären Prozesse
- Beispiele: Finanzbuchhaltung, Kostenrechnung, Personalwesen

Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

Wertschöpfungskettendiagramm: Gliederung von GP



Wertschöpfungskettendiagramm: Einbeziehung weiterer Elemente



Geschäftsprozesse: Ereignisgesteuerte Prozessketten (Grundelemente)

Symbol	Benennung	Bedeutung	Kanten-/ knotentyp
	Ereignis	Beschreibung eines eingetretenen Zustandes, von dem der weitere Verlauf des Prozesses abhängt	Ereignisknoten
	Funktion	Beschreibung der Transformation von einem Inputzustand zu einem Outputzustand.	Aktivitätsknoten
	„exklusives oder"	Logische Verknüpfungsoperatoren beschreiben die logische Verknüpfung von Ereignissen und Funktionen	Bedingungsknoten
	„oder"		
	„und"		
	Organisatorische Einheit	Beschreibung der Gliederungsstruktur eines Unternehmens	Organisationsknoten
	Informationsobjekt	Abbildung von Gegenstanden der realen Welt	Aktivitätsknoten
	Anwendungssystem	Anwendungssysteme zur Prozessunterstützung (z. B. SAPERP)	Aktivitätsknoten
	Kontrollfluss	Zeitlich-logischer Zusammenhang von Ereignissen und Funktionen	Kontrollflusskante
	Datenfluss	Beschreibung, ob von einer Funktion gelesen, geschrieben oder geändert wird.	Datenflusskante
	Zuordnung	Zuordnung von Ressourcen/ Organisatorischen Einheiten	Zuordnungs-beziehungskante

EPKs: Funktionen und Ereignisse

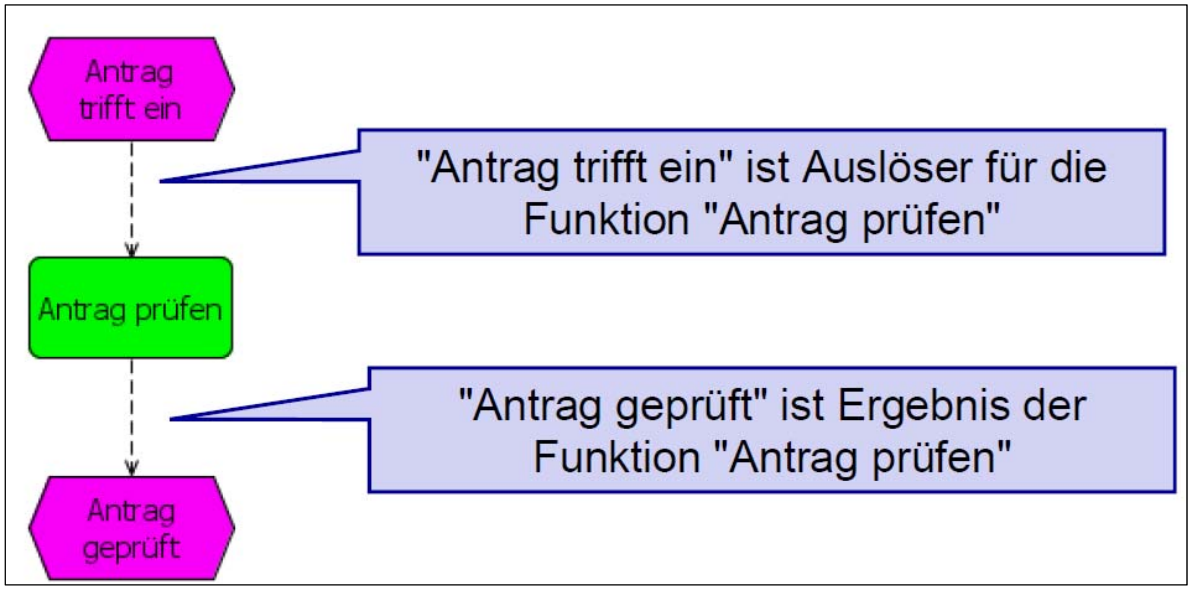
Funktionen: fachliche Tätigkeit, die einen Anfang und ein Ende hat



Ereignisse: Zustand zu einem bestimmten Zeitpunkt, kann Auslöser oder Ergebnis einer Funktion sein



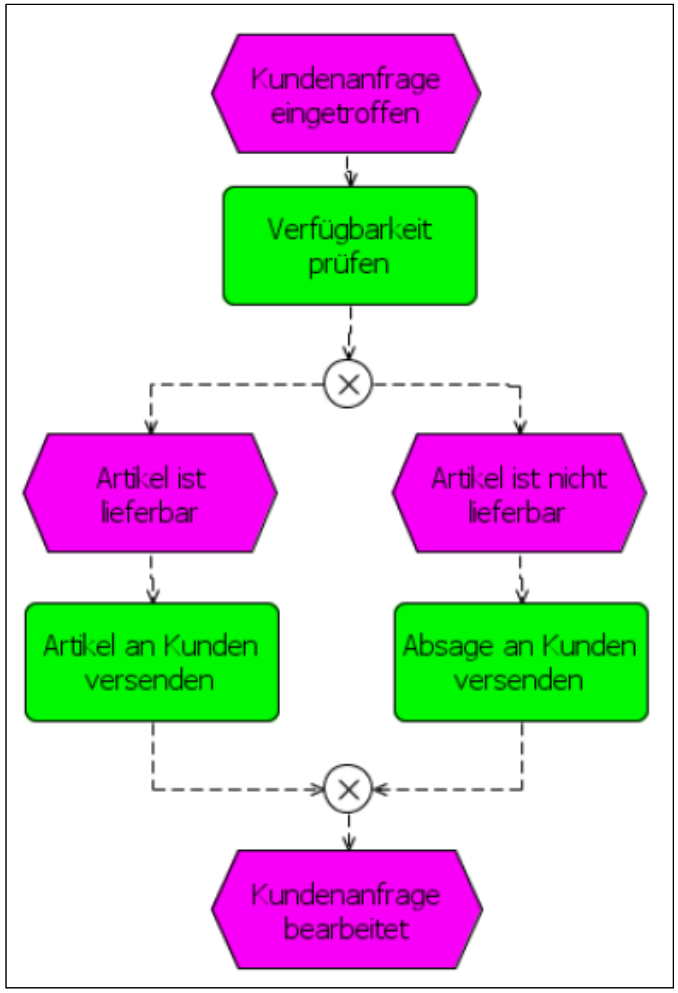
Aufeinanderfolge von Ereignissen und Funktionen: Pfeile zeigen die zeitlich-logische Reihenfolge zwischen Ereignissen und Funktionen an.



Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

EPKs: Modellierung von Alternativen
(XOR-Konnektor)

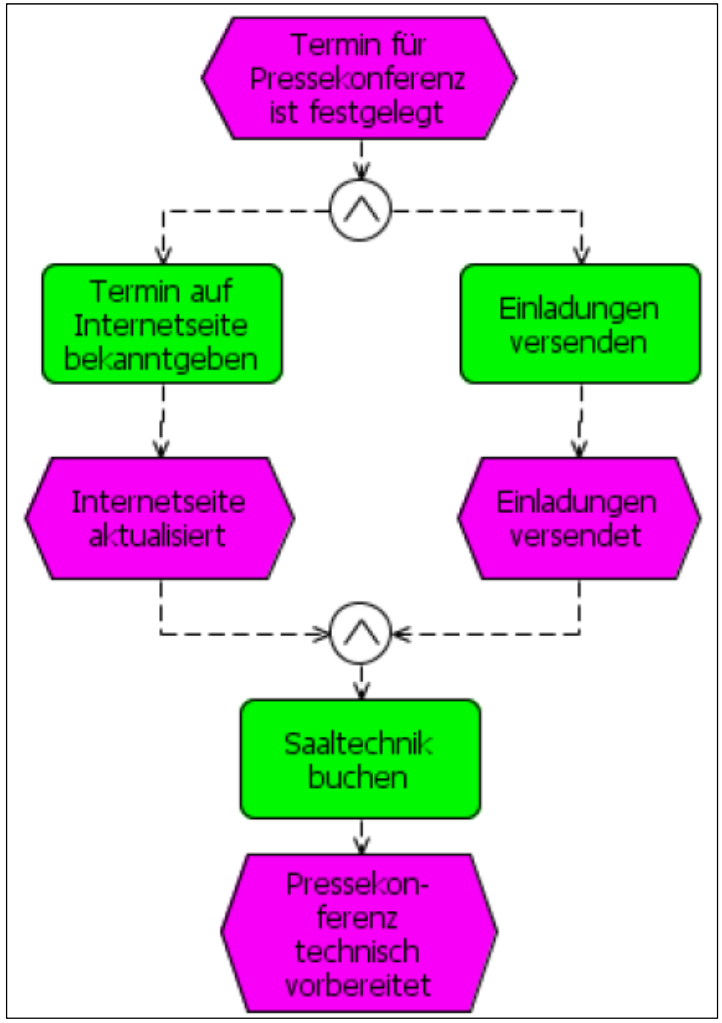
XOR-Konnektor modelliert eine Alternative: genau einer von mehreren möglichen Wegen wird genommen



Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

EPKs: Modellierung von Parallelität
(AND-Konnektor)

AND-Konnektor modelliert parallele Ausführung: mehrere Funktionen können gleichzeitig ausgeführt werden.

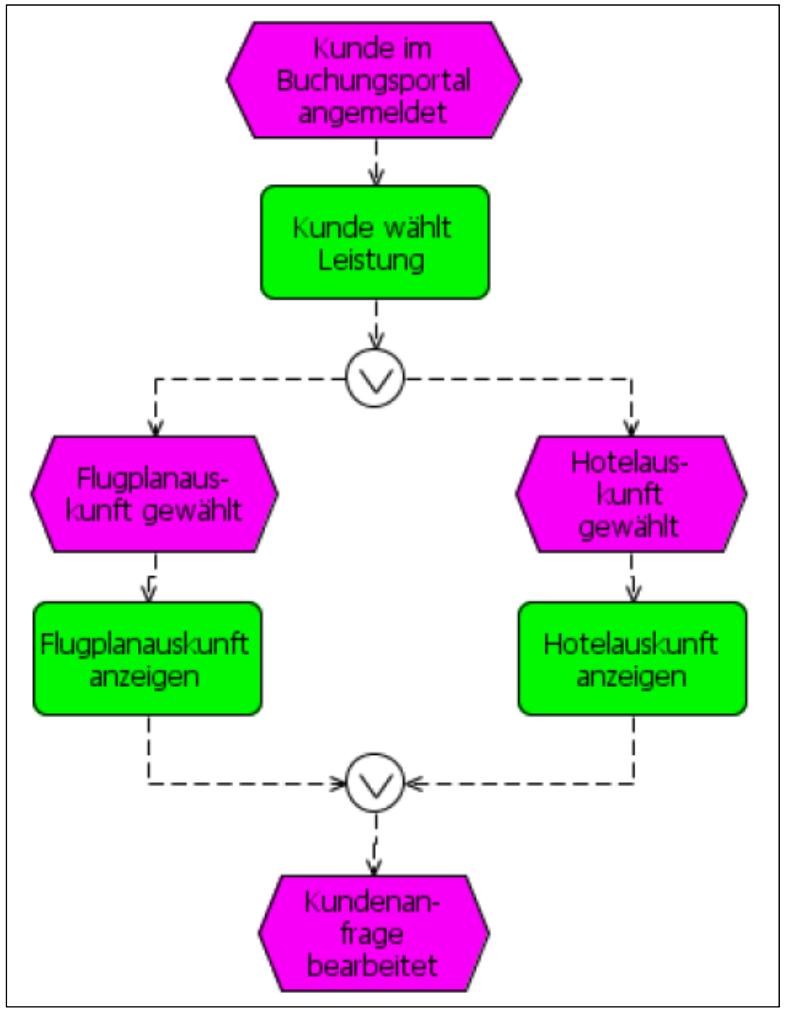


Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

EPKs: Modellierung von „ein oder mehere“ (OR-Konnektor)

OR-Konnektor modelliert die (parallele) Ausführung von Funktionen.

Mehrere der Möglichkeiten können parallel bearbeitet werden – aber mindestens eine muss bearbeitet werden.

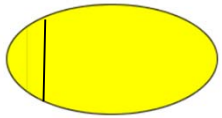


Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

GP-Modellierung: eEPK
(**e**rweiterte **E**reignisgesteuerte **P**rozess**k**etten)

eEPK ist eine EPK erweitert um:

- Organisationseinheiten (Zuständigkeiten):



Org-Einheit
(Abteilung, Referat)

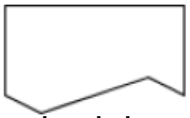


Stelle
(Sachbearbeiter)



Person

- Informationen/Material/Dokumente

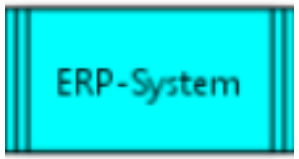


Papierdokument



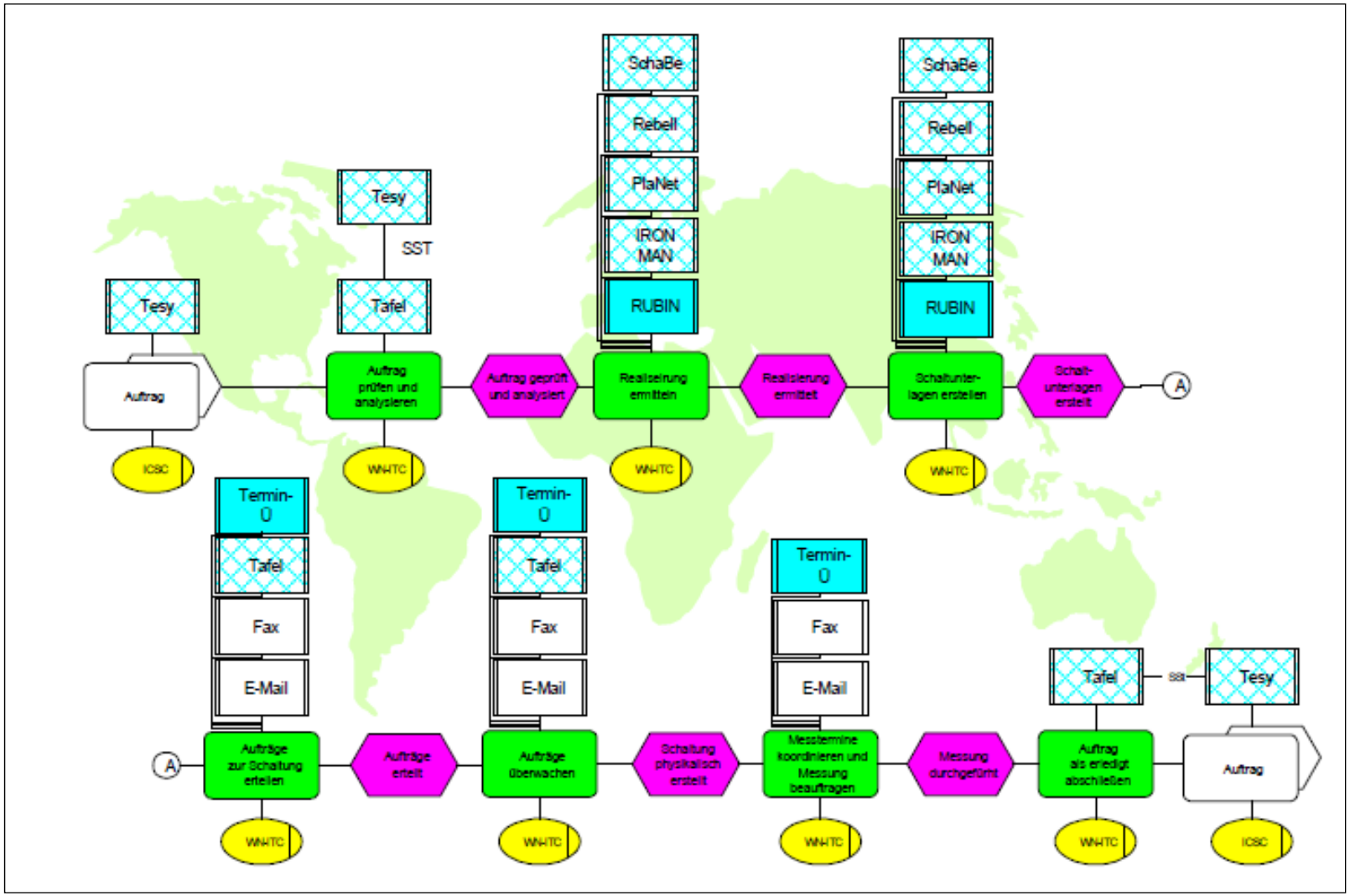
Datei

- Anwendungsysteme



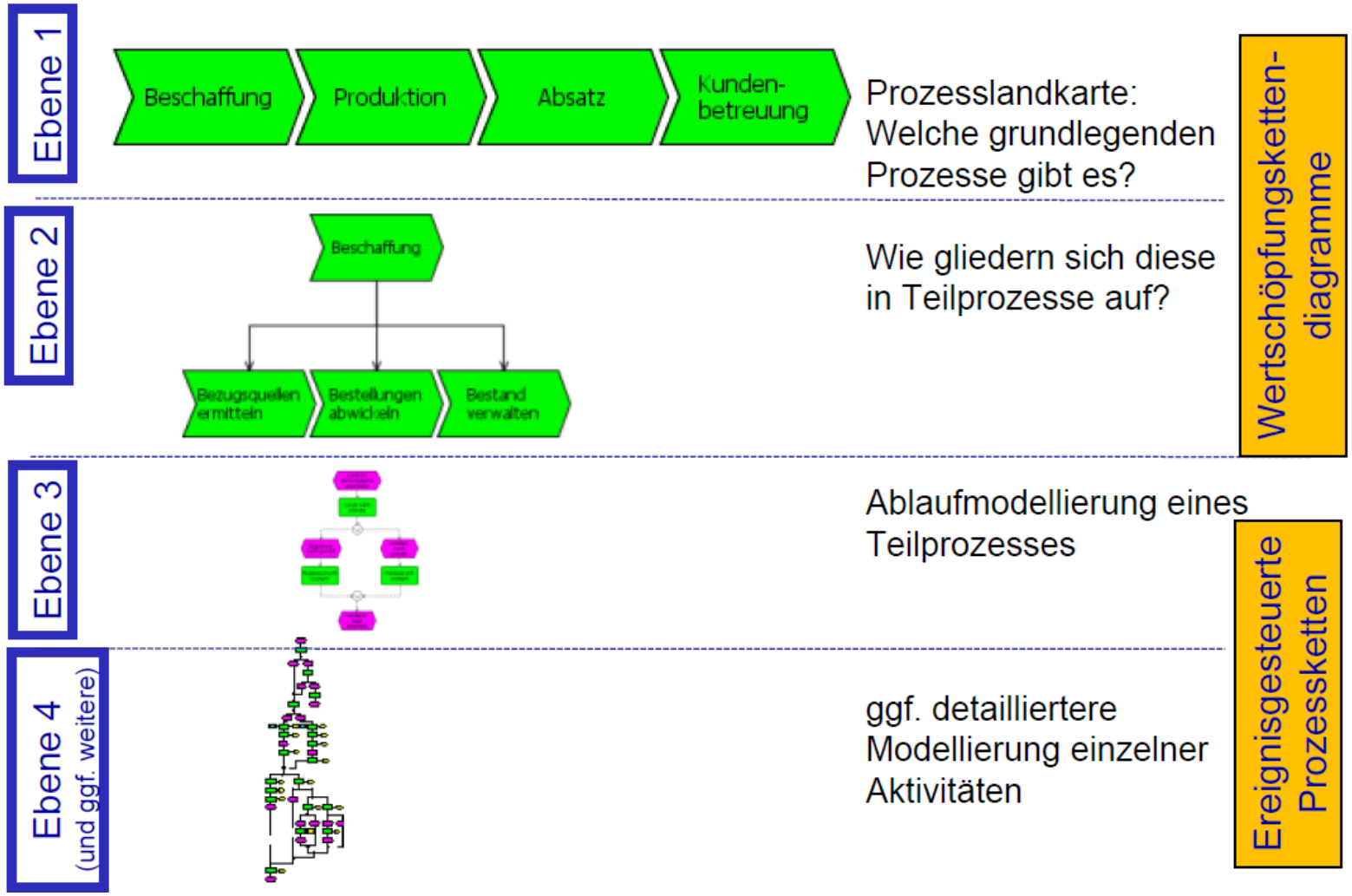
Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

eEPK: Komplexes Beispiel



Quelle: Deutsche Telekom AG

Geschäftsprozesse: Zusammenspiel WKD und EPK



Quelle: https://www.slideserve.com/urit/gesch-ftsprozessmodellierung-gpm-mit-ereignisgesteuerten-prozessketten-und-der-bflow-toolbox

GP- vs. SW-Modellierung

- **Ziele der GP-Modellierung** (EPKs oder auch BPMN)
  - orientiert an Unternehmensmodellen und Organisationsstrukturen
  - Identifizierung rollenbasierter Abläufe entlang der Wertschöpfungskette
  - Optimierung von Prozessen, Grundlage zur Automation
  - Funktions- und ereignisorientierte Darstellung der Abläufe

- **Ziele der SW-Modellierung** (SW-Engineering)
  - Modellierung eines Ausschnittes der realen Welt (Diskursbereich)
  - Use Case getriebene Modellierung
  - Architektur (Verschiedene Sichten auf das System)
  - Implementierung einer lauffähigen Applikation

Übung 2-1: GP-Modellierung
Aufgabenstellung

Muss

a) Entwerfen Sie für den Kino-Geschäftsprozess Wertschöpfungskettendiagramme + EPK's in mindestens 3 Ebenen (Level) wobei die letzte Ebene EPKs sein müssen:

- Das Lichtspieltheater muss über eine Produkt / Filmbeschaffung verfügen, dass die Auswahl und den Einkauf durchführt.
- Entsprechend wird eine Programmplanung realisiert, die entsprechende Werbeaktivitäten (Online, Prospekte, Kino-Zeitungen,..) durchführt.
- Online- und Kassenticketverkauf (incl. Abrechnung,…) müssen abgesichert werden.
- Der eigentliche Kinobetrieb ist technisch (Projektion,….) sowie verwaltungstechnisch (Ticketkontrolle, Rauminspektion, Reinigung, Reparatur,Finanzabrechnung …) durchzuführen.
- Der Kino Merchandising-Kiosk (Getränke, Knabber…., Werbung, Kinozeitschriften …..) muss realisiert und abgerechnet werden.

Muss

b) Mindestens 2 Gruppen präsentieren ihre Ergebnisse

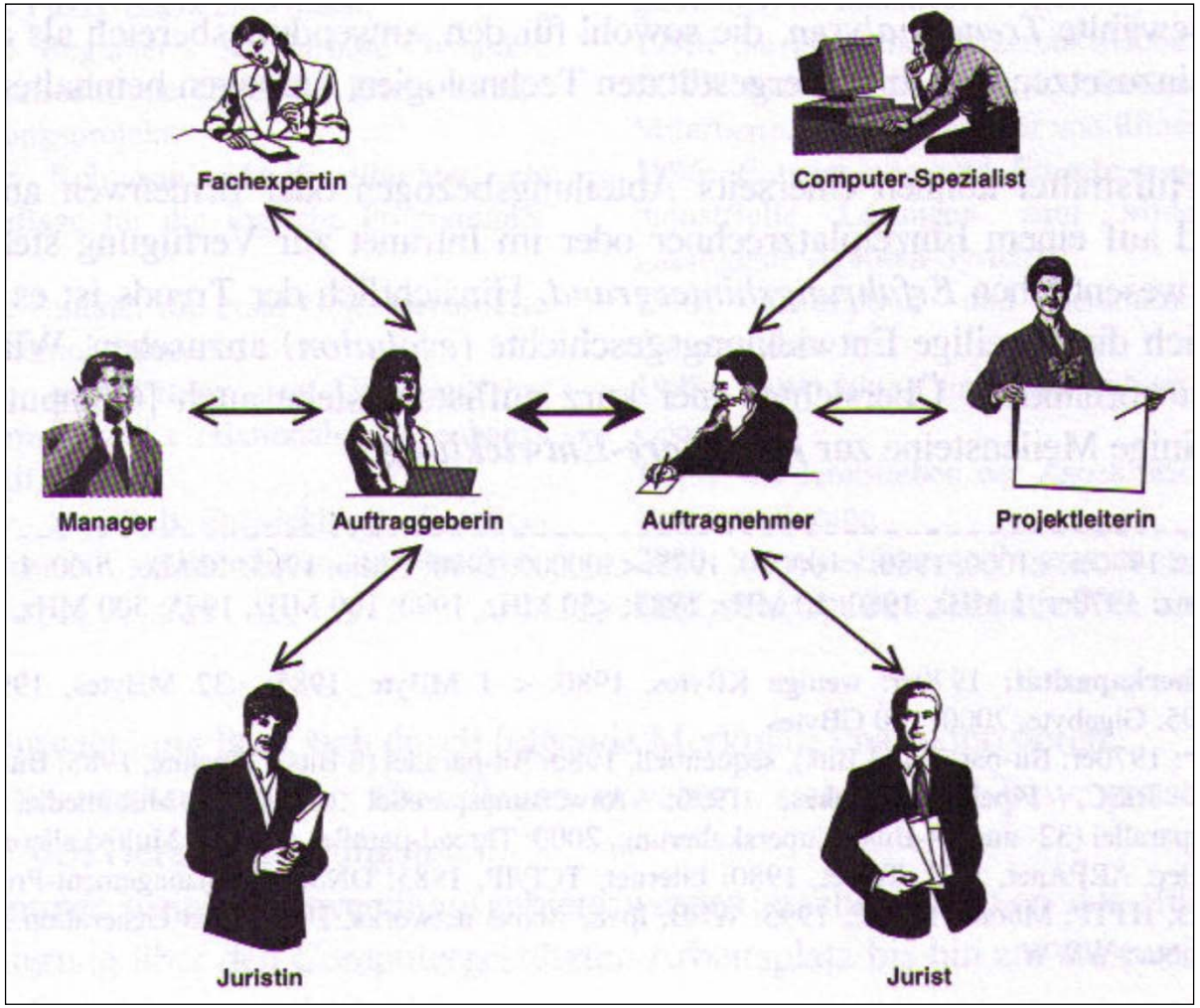
Motivation zur Anforderungsanalyse

Gründe fehlgeschlagener Projekte

- 13,1% unvollständige Anforderungen
- 12,4% unzureichende Integration der späteren Nutzer
- 10,6% unzureichende Ressourcen
- 9,9% unrealistische Erwartungen
- 9,3% unzureichende Bearbeitungsunterstützung
- 8,7% zu viele Änderungen der Anforderungen
- 8,1% Schwächen bei der Planung
- 7,5% System wurde bei der Einführung nicht mehr benötigt

Quelle: Standish Group, in Pfleeger, S.L.: Software Engineering, Prentice Hall, 1998

Problemdefinition als Basis



Quelle: Dumke, R.: Software Engineering, Vieweg-Verlag, 2003

Anforderungsanalyse: Einordnung und Definition

Bestandteile der Requirements Engineering:

- Anforderungserhebung (requirements elicitation),
- **Anforderungsanalyse** (requirements analysis),
- Anforderungsspezifikation (requirements specification) und
- Anforderungsbewertung (requirements validation)

Ziel der Anforderungsanalyse:

- die Anforderungen des Auftraggebers an das zu entwickelnde System zu ermitteln, zu strukturieren und zu prüfen.

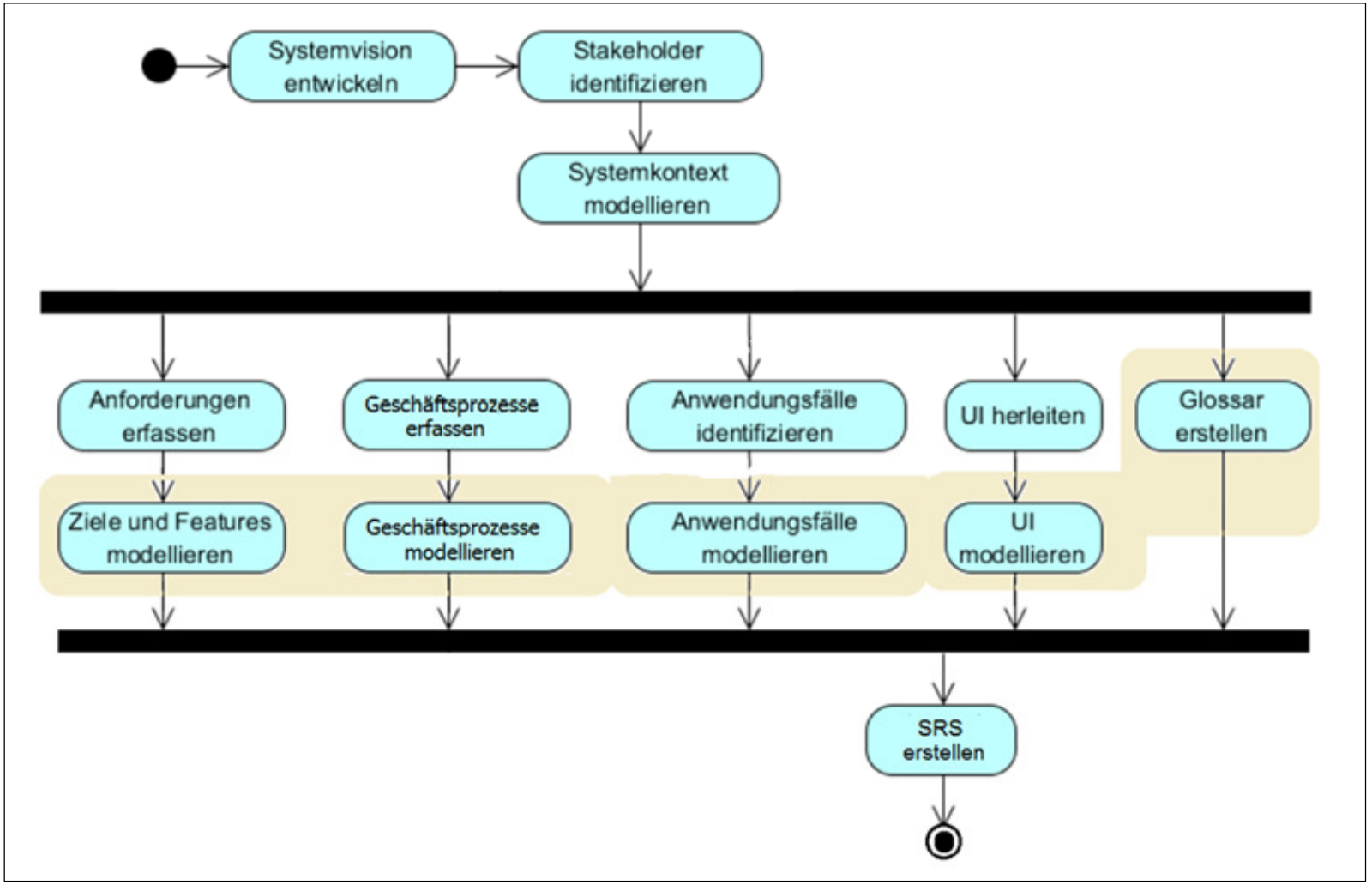
Ergebnis einer Anforderungsanalyse

- Das Ergebnis einer Anforderungsanalyse wird meistens in einem ***Lastenheft*** dokumentiert oder bei einer agilen Softwareentwicklung resultiert daraus ein ***Product Backlog***.

Anforderungsanalyse: Aufgaben

- Beschreibung der (fachlichen) *Anforderungen* an einem zu entwickelten Informationssystem (Softwaresystem) mit allen enthaltenen Komponenten.
- Aufteilung der Anforderungen in:
  - ***funktionale*** Anforderungen legen fest, was das fertige Projekt leisten soll
  - ***nicht-funktionale*** Anforderungen beschreiben, wie das Projekt die erwarteten Ziele erreichen soll. Sie werden auch oft in Qualitätsanforderungen und Rahmenbedingungen unterteilt.
- Alle funktionalen Anforderungen werden mit einer fortlaufenden Nummer, mit der jeweiligen Priorität und dem geplanten Realisierungstermin versehen und im Rahmen der Abnahme als Testkriterien verwendet.
- Festlegung der Grundfunktionen des Systems, der zu nutzende Standards (z.B. Schnittstellenformate), aber auch der zu unterstützende Laufzeitumgebungen.

Anforderungsanalyse: Ablauf



Quelle: Ebert, Christoph: Systematisches Requirements Engineering, Kapitel 5

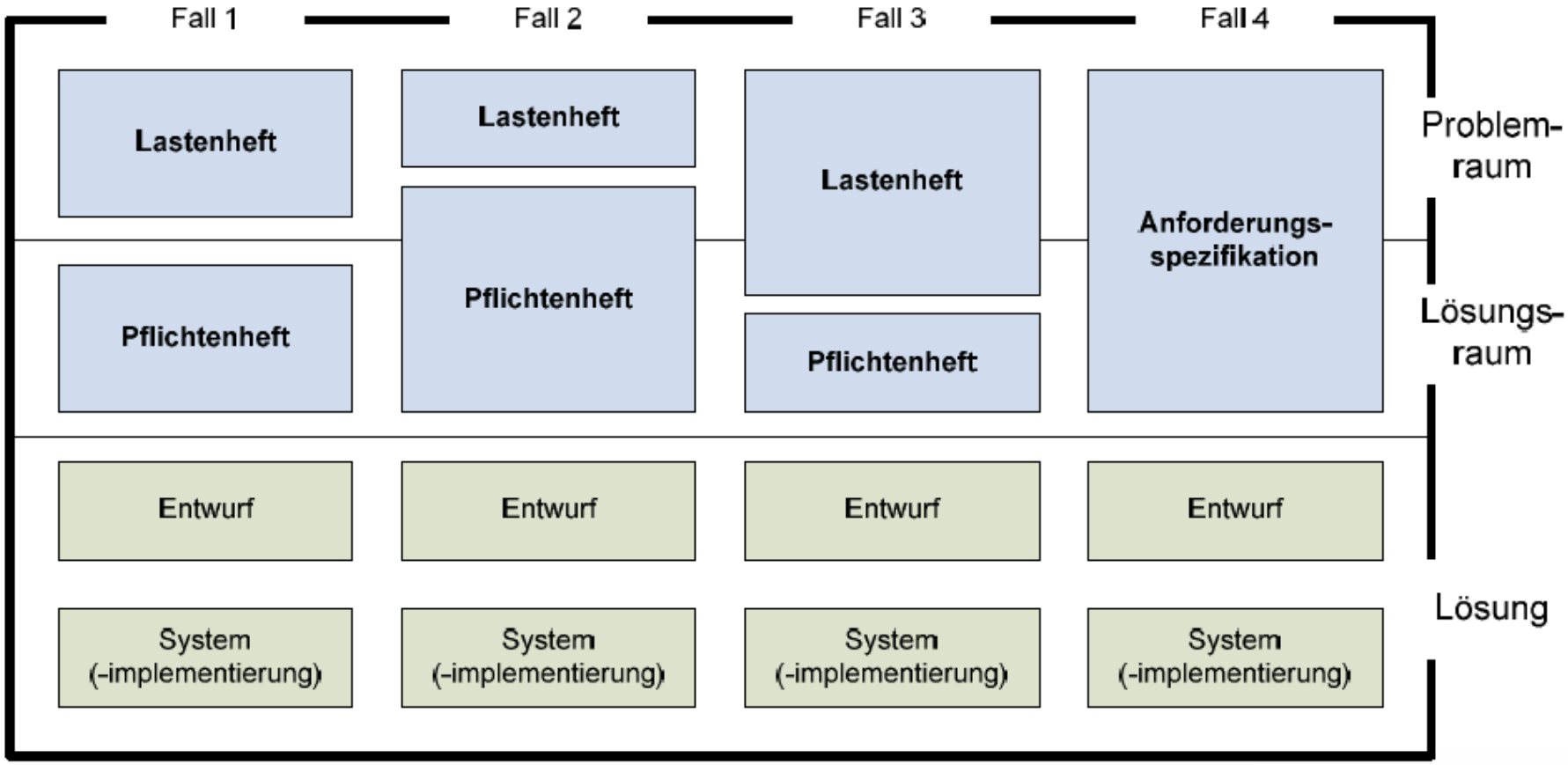
Qualität von Anforderungen

- Korrekt – vom Auftraggeber als zutreffend und richtig erkannt
- Konsistent – keine Mehrdeutigkeiten & gegensätzliche Eigenschaften
- Vollständig – mögliche Zustände des Systems sind dargestellt
- Realistisch – Schwierig in frühen Phasen einzuschätzen!
- Sinnvoll – Nutzerbedürfnisse (keine vergoldeten Anforderungen)
- Prüfbar – Realisierung muss nachweisbar sein – messbar!
- Verfolgbar – im Rahmen der Entwicklungsphasen

Quelle: Dumke, R.: Software Engineering, Vieweg-Verlag, 2003

Inhalte und Aufbau eines Pflichtenheftes

Varianten Anforderungsdokumentation



In Anlehnung an: Hammerschall, U.; Beneken, G.: Software Requirements, Pearson Deutschland GmbH, München 2013

Pflichtenheft: Überblick

- Aufgabe – Zusammenfassung aller fachlichen Anforderungen
- Adressaten – Auftraggeber & Auftragnehmer
- Inhalt – Daten, Funktionen, Leistung & Qualität (WAS, nicht WIE)
- Form – Standardisiertes Format (grobes Gliederungsschema)
- Sprache – detaillierte verbale Beschreibung (inkl. Nummerierung)
- Didaktik – gut lesbar → leichte Einarbeitung
- Zeitpunkt – Versionierung (Anforderungskonferenzen AG ←→ AN)
- Umfang – ausreichendes Detaillierungsniveau ist anzustreben

Quelle: Balzert, H.: Lehrbuch der Software-Technik, Spektrum Akademischer Verlag, 1996

Pflichtenheft: Elemente

1 Zielbestimmung
- Ziele, keine Funktionen
- 1.1 Muss-Kriterien
- 1.2 Kann-Kriterien Möglichkeit zum Verzicht
- 1.3 Abgrenzungskriterien
- Ziele, die nicht erreicht werden sollen

2 Einsatz
- 2.1 Anwendungsbereiche
- 2.2 Zielgruppen
- 2.3 Betriebsbedingungen
- physikalische Umgebung, Betriebszeit, Beobachtung vs. unbeaufsicht. Betrieb

3 Umgebung
- Voraussetzungen zur Installation/zum Betrieb
- 3.1 Software
- 3.2 Hardware
- 3.3 Orgware

4 Funktionalität
- typische Arbeitsabläufe, keine Verwaltungsfkt.

5 Daten
- langfristig zu speichernde Daten, Umfang

6 Leistungen
- z.B. zeitliche Anforderungen

7 Benutzungsoberfläche
- Eigenschaften der Nutzer, Art der Nutzung

siehe Beispieldatei „Pflichtenheft.doc“

Übung 2-2: Pflichtenheft
Aufgabenstellung

Muss

a) Erstellen bzw. Überprüfen Sie das Pflichtenheft ihres im Rahmen der Vorlesung „Objektorientierte Programmierung“ realisierten Projektes.

- Analysieren Sie das Ihnen übergebene Beispiel eines Pflichtenheftens
- Führen Sie in ihrer Gruppe ein Brainstorming zu den potentiellen Anforderungen ihres Projektes durch, berücksichtigen Sie dabei die Ihnen bereits bekannte Methode der Metaplantechnik.
- Übertragen Sie die gewonnen Erfahrungen auf das eigene Projekt und erarbeiten Sie ein erstes grobes Pflichtenheft
- Erstellen Sie eine entsprechende Dokumentation (z.B. Power Point) zu den erarbeiteten Ergebnissen

Muss

b) Mindestens 2 Gruppen präsentieren ihre Ergebnisse

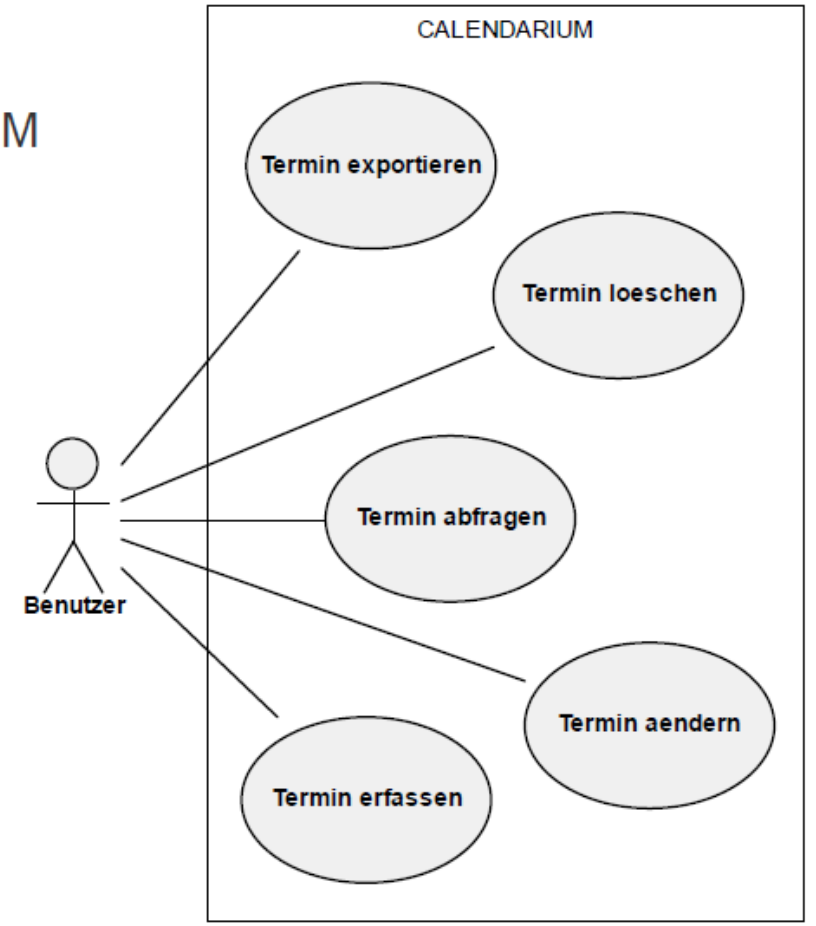
Ableitung und Untersetzung von Use Cases

Use Case Diagramm: Grundlagen (1/2)

- **Use Cases** (Anwendungsfälle) beschreiben das **Verhalten**, das von dem zu entwickelnden System **erwartet** wird.
- Graphische Darstellung der Use Cases: **Use Case Diagramm** - zeigt Beziehungen zwischen Akteuren und Anwendungsfällen auf.
- **Akteure** (verschiedene Stereotypisierung)
  - Person im Sinne einer konkreten Rolle innerhalb einer Anwendung
  - Technische Systeme und deren Rolle
  - Externes System mit der Auslösung eines zeitlichen Ereignisses
- **Use Cases**
  - Menge von Aktivitäten eines Systems aus Nutzersicht
  - Komplette unteilbare Beschreibung
  - Zu jedem Anwendungsfall existiert eine textuelle Beschreibung
- **Use Case Beziehungen:**
  - Assoziation
  - Vererbung
  - <<include>> und <<extend>>

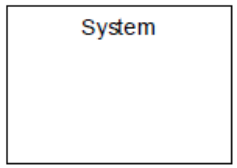
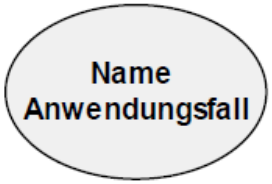
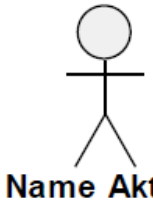
Use Case Diagramm: Grundlagen (2/2)

- **System** (was wird beschrieben?)
  - der Onlinekalender CALENDARIUM
- **Akteur** (wer benutzt das System?)
  - Benutzer des Kalenders
- **Anwendungsfälle des Benutzers** (was machen die Akteure?)
  - Abfragen von Terminen
  - Exportieren von Terminen
  - Löschen von Terminen
  - Ändern von Terminen
  - Erfassen von Terminen



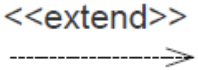
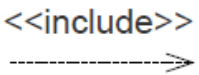
Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Use Case Diagramm: Grundlelemente

Name	Syntax	Beschreibung
Systemgrenze		Grenze zw. dem eigentlichen System und den Benutzern des Systems
Anwendungsfall		vom System erwartetes Verhalten
Akteur		Rolle der Systembenutzer

Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Use Case Diagramm: Beziehungen

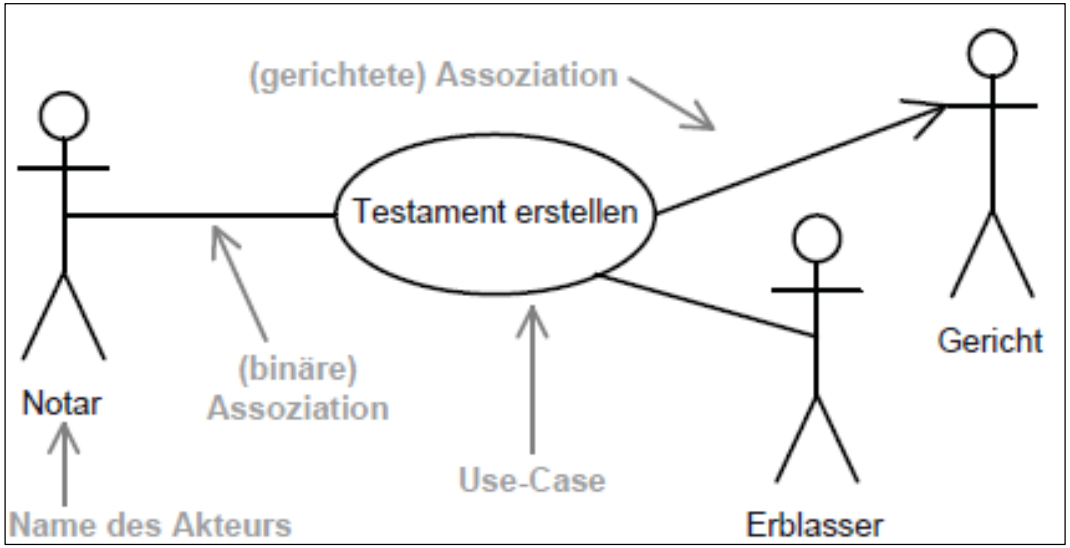
Name	Syntax	Beschreibung
Assoziation		Beziehung zwischen Anwendungsfällen und Akteuren
Generalisierung		Vererbungsbeziehung von Anwendungsfällen und Akteuren
extend		*A extends B*: opt. Verwenden von Anwendungsfall A durch Anwendungsfall B
include		*A includes B*: notw. Verwenden von Anwendungsfall B durch Anwendungsfall A

Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Use Cases: Assoziation

Die Beteiligung eines Akteurs an einem Use-Case-Ablauf wird durch eine **Assoziation** (allg. Beziehung) zwischen dem Akteur und dem Use-Case dargestellt.

- Die Assoziationen müssen binär sein, das bedeutet, dass an einer Assoziation genau zwei Partner beteiligt sind.
- Um den Ein- und Ausgabefluss darzustellen, dürfen die Assoziationen zudem gerichtet sein.

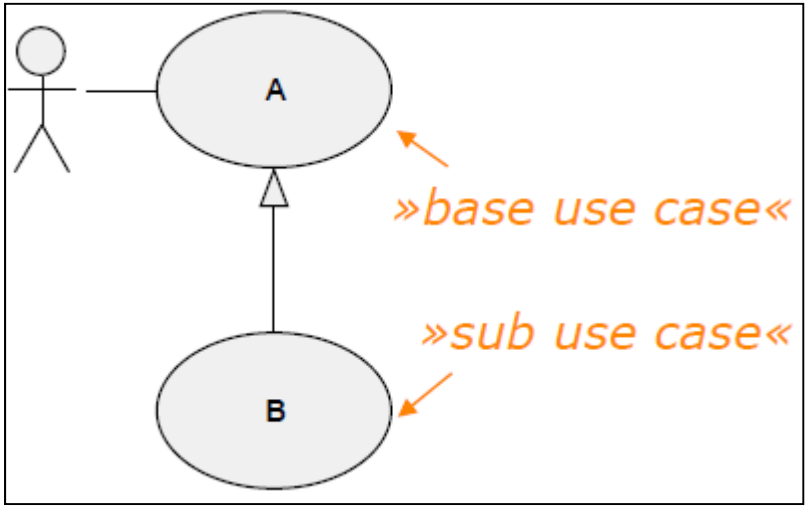


Quelle: Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012, S.252

Use Cases: Generalisierung bei Anwendungsfällen

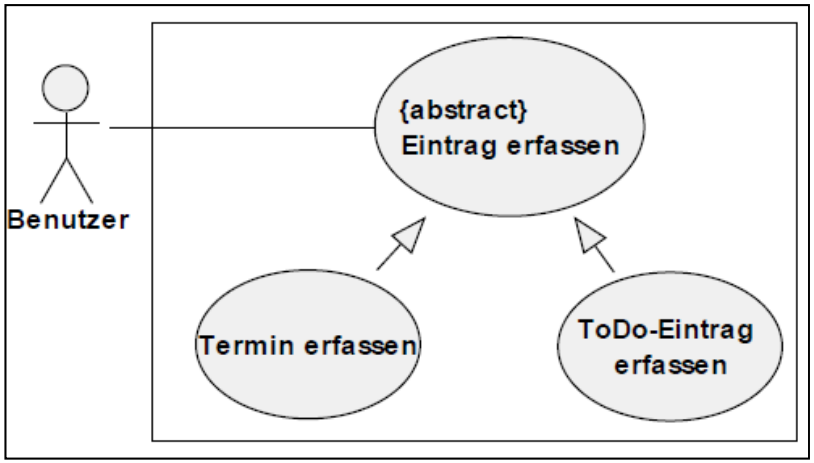
Generalisierung bei Anwendungsfällen:

- B erbt das Verhalten von A und kann dieses überschreiben oder ergänzen
- B erbt alle Beziehungen von A
- B benötigt A (übernimmt Grundfunktionalität von A)
- B entscheidet, was von A ausgeführt bzw. geändert wird



Modellierung abstrakter Anwendungsfälle möglich ({abstract}):

- abstrakte Anwendungsfälle sind nicht ausführbar!

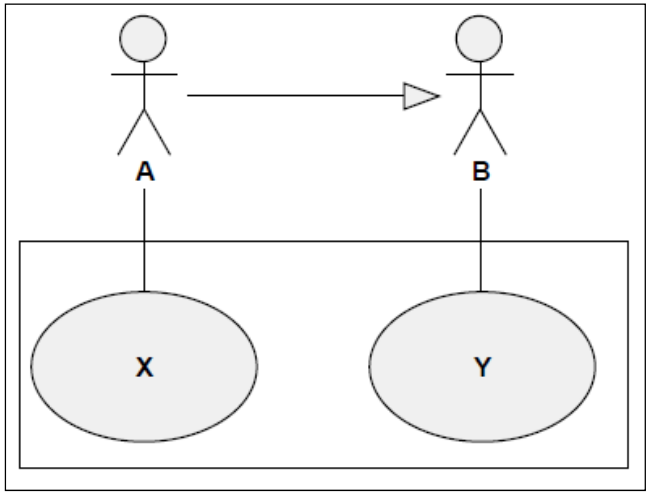


Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Use Cases: Generalisierung bei Akteuren

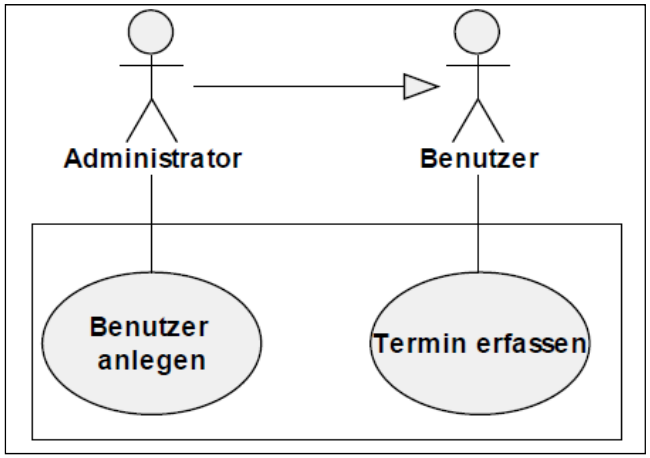
Generalisierung bei Akteuren:

- Akteur A erbt von Akteur B
- A kann mit den Anwendungsfällen X und Y kommunizieren
- B kann nur mit Y kommunizieren



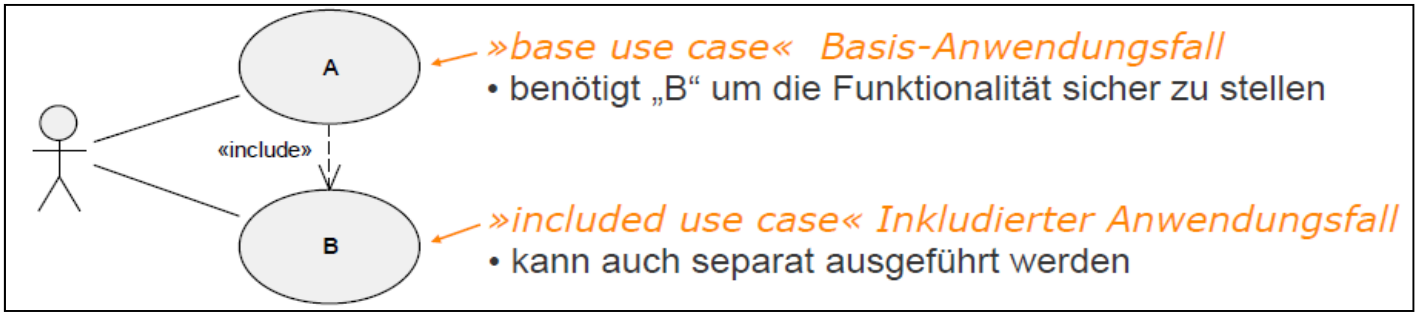
Besonderheiten:

- Mehrfachvererbung ist erlaubt
- Unterscheidung, ob mehrere Akteure gemeinsam mit einem Anwendungsfall kommunizieren können oder müssen.



Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

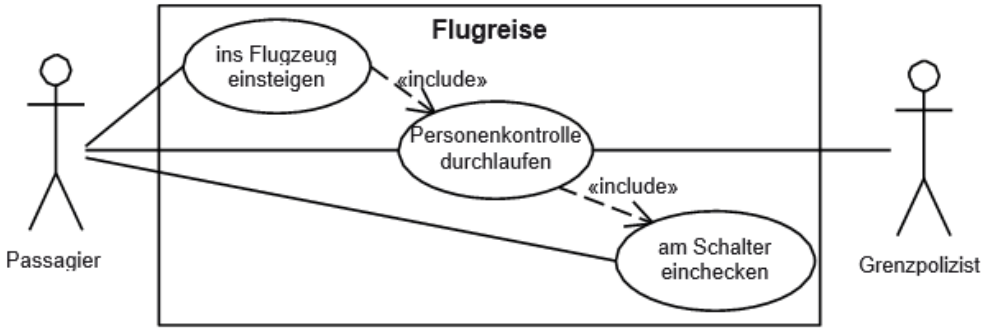
Use Cases: Beziehung <<include>>



<<include>>:

- Das **Verhalten** des benutzten Anwendungsfalls (inkludierter Anwendungsfall) wird in den benutzenden Anwendungsfall (Basis-Anwendungsfall) **eingebunden.**
- D.h. innerhalb eines Anwendungsfalles kommt ein anderer vor.

→ Vermeidung redundanter Darstellungen

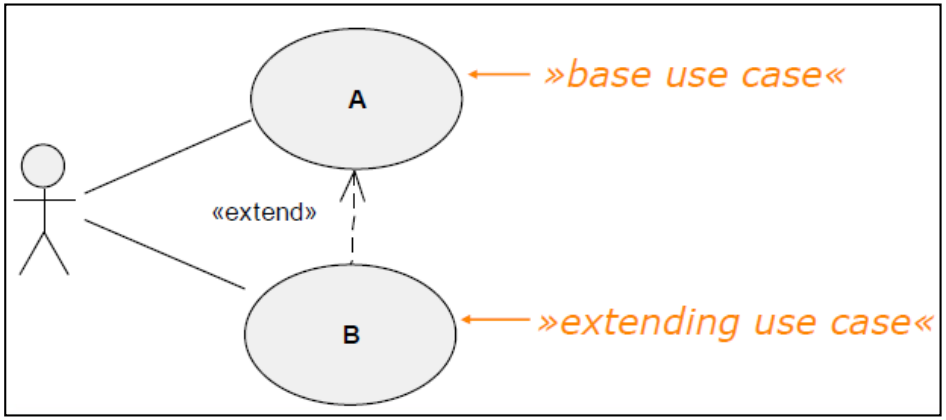


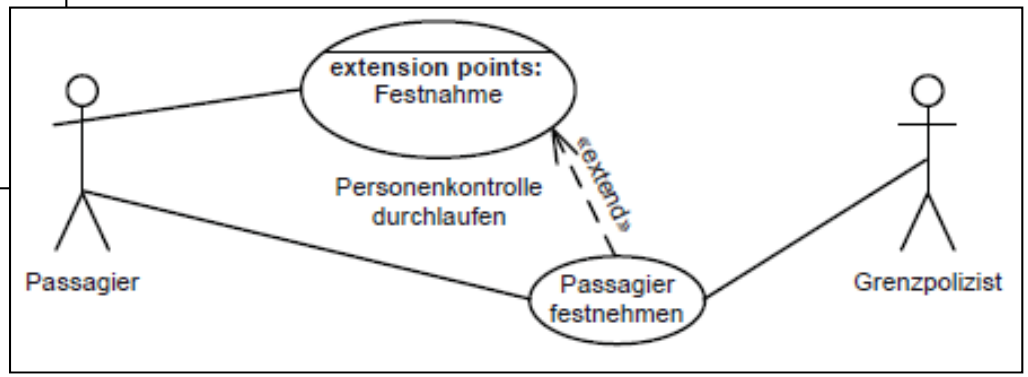
Quellen: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012
Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012,

Use Cases: Beziehung <<extend>> (1/2)

<<extend>>:

- Das **Verhalten** eines Use-Case (A) **kann** durch einen anderen Use-Case (B) **erweitert** werden, aber **nicht muss**.
- A bzw. B können auch separat ausgeführt werden.
- Angabe des »Wo« durch Erweiterungsstellen (extension points) in A.
- Angabe des »Wann« durch Bedingung in A bzw. als Teil der «extend»-Beziehung.

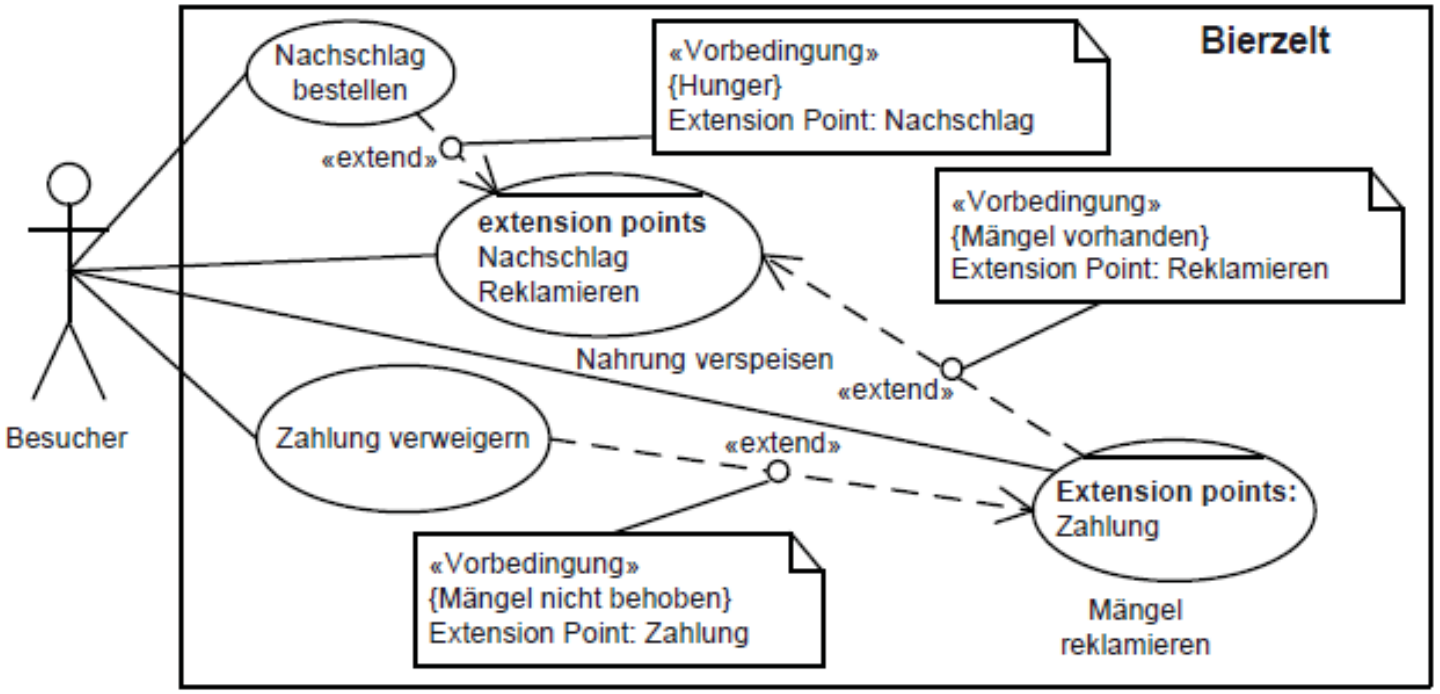




Quellen: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012
Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012,

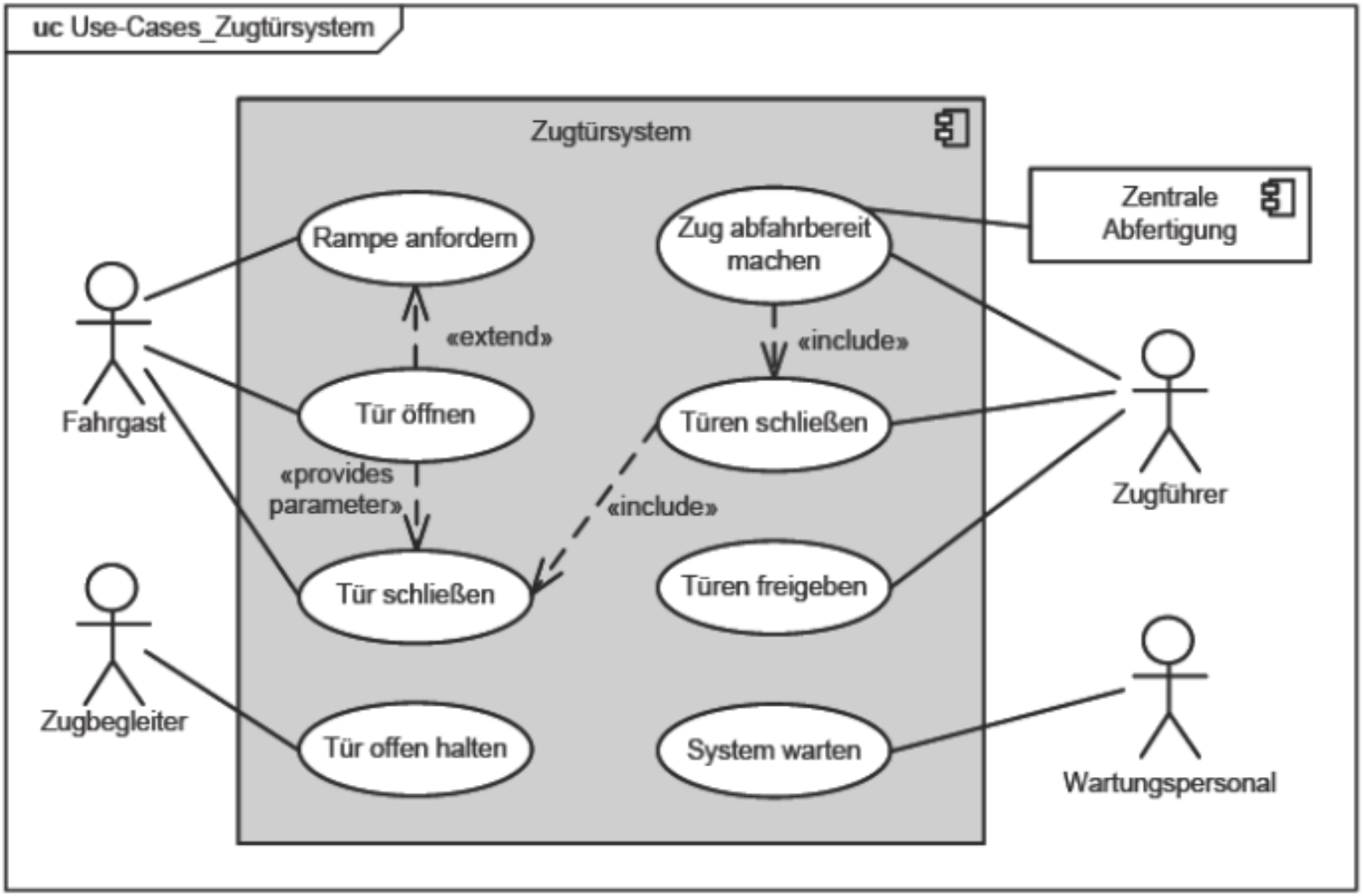
Use Cases: Beziehung <<extend>> (2/2)

Mehrfache Erweiterung des Use-Cases „`Nahrung verspeisen`“ und die Erweiterung des erweiternden Use-Cases „`Mängel reklamieren`“ durch den Use-Case „`Zahlung verweigern`“.



Quelle: Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012, S.261

Use Case Diagramm: Beispiel Zugtürsystem



Quelle: Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012, S.48

Use Cases: Spezifikation

Je **Ellipse** existiert ein Text, der den Anwendungsfall genauer beschreibt

- Akteur (Rollen)
- Vor- und Nachbedingungen sowie Invarianten
- Nicht-funktionale Anforderungen
- Ablaufbeschreibung
- Ausnahmen und Fehlersituationen
- Variationen & Regeln
- Services
- Ansprechpartner
- Änderungshistorie (Versionierung)
- Anmerkungen und referenzierte Diagramme

Quelle Abb. rechts: https://templatelab.com/wp-content/uploads/2016/03/Use-Case-Template-03.jpg

Use Case # [number: name]

GENERAL CHARACTERISTICS	
Intent	[a summary statement of the purpose of the use case]
Scope	[one of: company, name of system under design being considered black box, or another as appropriate] <[owning use case package]>
Level	[one of: business level , system level, or component]
Author	[name of use case author(s)]
Last Update:	[date last updated / change history]
Status	[one of : incomplete, under review, finalized, etc.] [one of: conceptualization, analysis, etc.]
Primary Actor	[role name for the primary actor, <optional description>]
Secondary Actors	[role names of other actors (could be systems) relied upon to accomplish use case]
Preconditions	[what we expect is already the state of the world] <[pointer to precondition object model]>
<Dynamic Preconditions>	<[list of Use Cases # that must have run successfully | scenario diagrams that must be previously run]>
Assumptions	[any assumptions relative to this use case]
Trigger	[the event that starts the use case]
Success Post Condition	[the state of the world upon successful completion] <[pointer to post condition object model]>
Failed Post Condition	[the state of the world if use case abandoned] <[pointer to post condition object model]>
< Models>	<[pointer to Use Case-specific Models, if any]>
Operations Concepts	[a description of the approach(s) to be taken by the implementation of this case] <[pointers to visualizations]>
Overview	[description in words that encompasses all scenarios]

Klausur relevant

Übung 2-3: Use Case-Modellierung
Aufgabenstellung

a) Entwerfen Sie für den Kino-Geschäftsprozess ein Use Case Diagramm:

- Das Lichtspieltheater muss über eine Produkt / Filmbeschaffung verfügen, dass die Auswahl und den Einkauf durchführt.
- Entsprechend wird eine Programmplanung realisiert, die entsprechende Werbeaktivitäten (Online, Prospekte, Kino-Zeitungen,..) durchführt.
- Online- und Kassenticketverkauf (incl. Abrechnung,…) müssen abgesichert werden.
- Der eigentliche Kinobetrieb ist technisch (Projektion,….) sowie verwaltungstechnisch (Ticketkontrolle, Rauminspektion, Reinigung, Reparatur,Finanzabrechnung …) durchzuführen.
- Der Kino Merchandising-Kiosk (Getränke, Knabber…., Werbung, Kinozeitschriften …..) muss realisiert und abgerechnet werden.

b) Mindestens 2 Gruppen präsentieren ihre Ergebnisse

Muss

Übung 2-4: Use Case Diagramm
Aufgabenstellung

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

a) Erstellen Sie ein äquivalentes Use Case Diagramm für das in Übung 2-2 erstellte Pflichtenheft für die Projektaufgabe, das die wichtigsten Anwendungsfälle und ihre Beziehungen enthält.

- Identifikation der beteiligten Akteure
- Identifikation der Anwendungsfälle und potentieller Beziehungen
- Spezifizieren der Anwendungsfälle entsprechend der Schablone
- Dokumentieren Sie ihr Projektergebnis zur weiteren Verwendung

b) Mindestens 2 Gruppe präsentieren ihre Ergebnisse

Verfeinerung von Use Cases

- Bislang hat jeder **Use Case** einen **Namen**, **Verbindungen** zu Akteuren und weiteren UCs und eine **Spezifikation** → diese Information genügt nicht, um mit der Realisierung des Systems zu starten → UCs müssen weiter verfeinert werden!
- **UML-Mitteln zur Verfeinerung** eines Use Cases
  - Aktivitätsdiagramme: beschreiben den Ablauf eines UC
  - Zustandsdiagramme: modellieren die Zustände des Systems während des UC-Verlaufes
  - Sequenzdiagramme: stellen die Interaktionen zwischen den Benutzern und dem System dar.
- Im Allgemeinen reichen ein UC-Diagramm (einschl. UC-Spezifikation) und **Aktivitätsdiagramme** zur präzisen Beschreibung eines UC aus.
  - Zustandsdiagramme werden in der Analysephase zur detaillierten Beschreibung von Objekten verwendet.
  - Sequenzdiagramme benötigt man zur exemplarischen Darstellung von Abläufen (Interaktionen) zwsichen Objekten →können schnell unübersichtlich werden!

Aktivitätsdiagramm: Grundlagen (1/7)

- **Aktivitätsdiagramme** dienen der Beschreibung eines **Kontroll-** und **Datenflusses** (vgl. Flow Chart – Programmablaufplan)

- Aktivität (ab UML Version 2.0 wurde der Begriff „Aktion“ eingeführt!). Eine **Aktivität** ist ein gerichteter Graph:
  - Knoten: **Aktionen** oder selbst wieder Aktivitäten
  - Kanten: **Kontroll-** und **Datenflüsse**

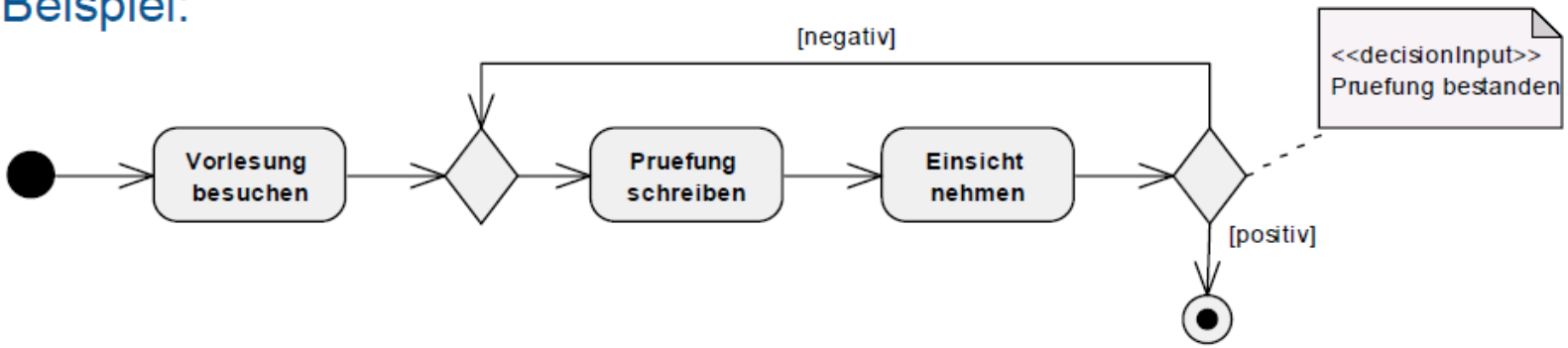
- **Aktionen** sind elementare Bausteine für beliebiges benutzerdefiniertes Verhalten (Sie beschreiben einen Schritt im Prozess).
  - atomar, können aber abgebrochen werden (zeitverbrauchend!)
  - können Eingabewerte zu Ausgabewerten verarbeiten

- **Kanten** verbinden Knoten und legen mögliche Abläufe einer Aktivität fest
  - **Kontrollflusskanten**: Drücken eine reine Kontrollabhängigkeit zwischen Vorgänger- und Nachfolgerknoten aus
  - **Objektflusskanten**: Transportieren zusätzlich Daten und drücken dadurch auch eine Datenabhängigkeit zwischen Vorgänger- und Nachfolgerknoten aus

Aktivitätsdiagramm: Grundlagen (2/7)

- **Token** = „virtueller Koordinationsmechanismus“ zur Beschreibung von Aktivitätsabläufen
  - beschreibt möglichen Ablauf einer Aktivität zur Laufzeit
  - fließen entlang der Kanten von Vorgänger- zu Nachfolgerknoten
- **Alternative Abläufe / Entscheidungsknoten** - definiert alternative Zweige und repräsentiert eine »Weiche« für den Tokenfluss
- **Alternative Abläufe / Vereinigungsknoten -** führt alternative Abläufe wieder zusammen. Token werden, sobald möglich, an den Nachfolgerknoten weitergereicht.

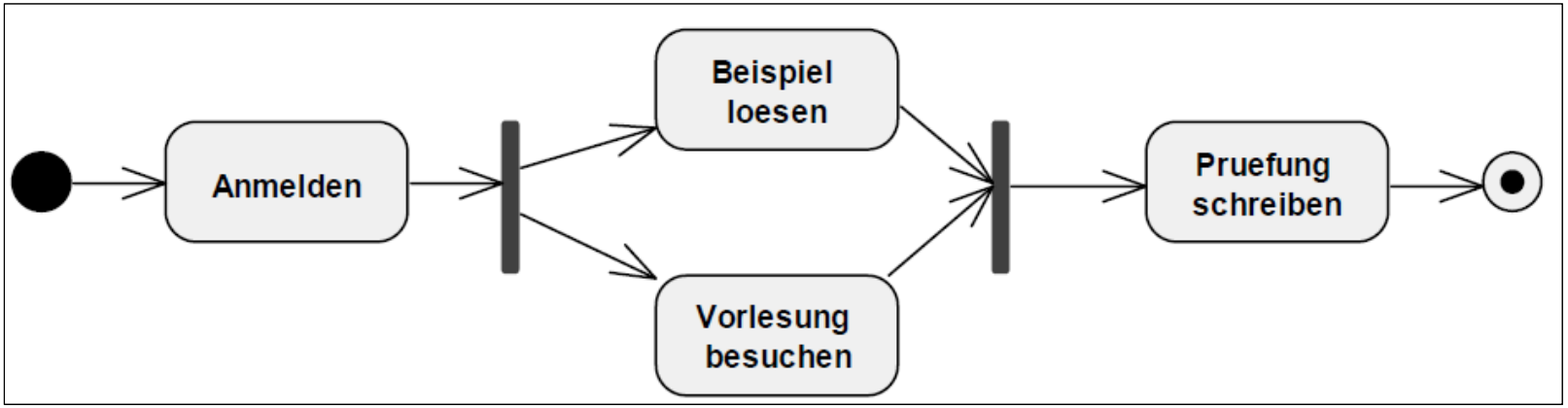
Beispiel:



Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Grundlagen (3/7)

- **Nebenläufige Abläufe - Parallelisierungsknoten** dient der Aufspaltung von Abläufen
  - eingehende Token werden für alle ausgehenden Kanten dupliziert, sobald zumindest eine Überwachungsbedingung diese akzeptiert
- **Nebenläufige Abläufe - Synchronisierungsknoten** führt nebenläufige Abläufe zusammen
  - Kontrolltoken verschiedener Kanten werden vereinigt und nur ein einzelnes Token weitergereicht



Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Grundlagen (4/7)
Daten-Token, Objektflüsse und **Objektknoten**

- Neben den Kontroll-Token, deren Hauptaufgabe das Anregen von Aktionsabläufen ist, gibt es **Daten-Token**. Diese dienen als Transportvehikel, auf denen Daten oder Werte durch das Diagramm getragen werden. Solche Token bewegen sich über Kanten, an denen mindestens ein **Objektknoten** beteiligt ist.
- In einen Objektknoten hineingehende Token (1) repräsentieren Daten oder Werte, die in dem Objektknoten gesetzt bzw. gesammelt oder durch den Objektknoten repräsentiert werden.
- Aus einem Objektknoten herausgehende Token transportieren das Objekt selbst bzw. Daten des Objekts in die Folgeaktion (2).
- Der Objektknoten ist daher ein **Notationsmittel**, um den Fluss von Objekten und Werten zu modellieren, der Knoten an sich stellt kein Objekt bzw. keinen Datenwert dar.



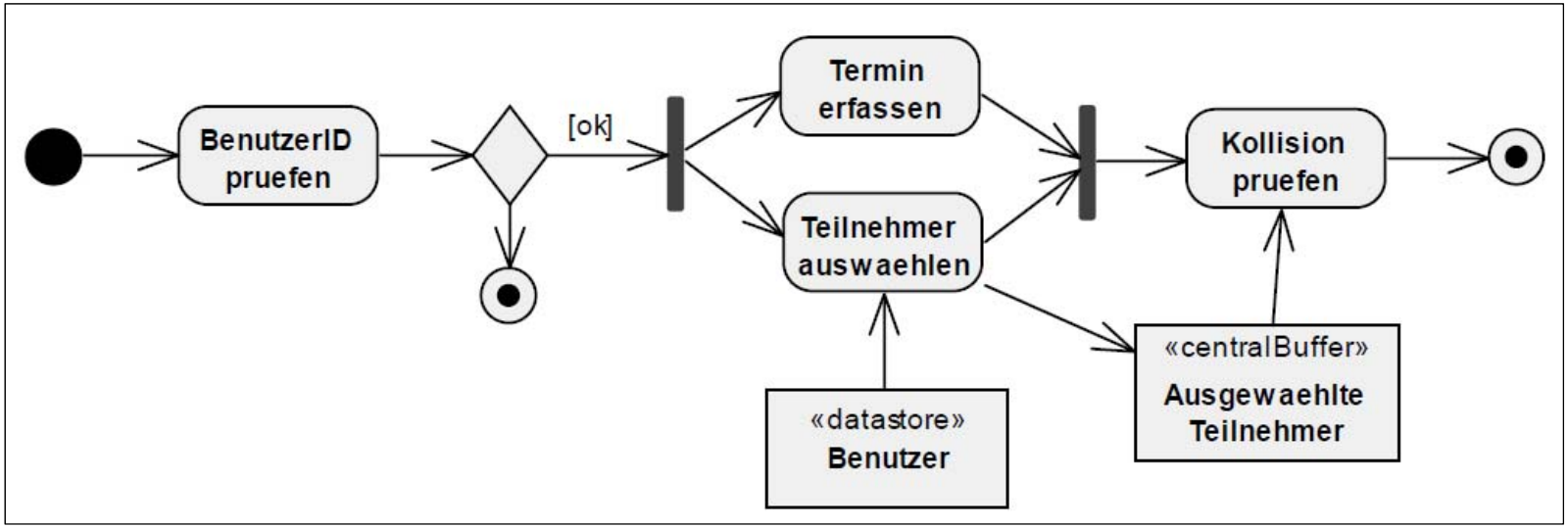
Quelle: Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012, S.48

Aktivitätsdiagramm: Grundlagen (5/7)

Objektflüsse und **Objektknoten: Beispiel Pufferknoten**

Spezielle Objektknoten sind die sog. **Pufferknoten**:

- **central buffer**: Token werden aus dem Puffer entnommen und weitergegeben
- **data store:** vom Token wird eine lokale Kopie angelegt, die weitergegeben wird



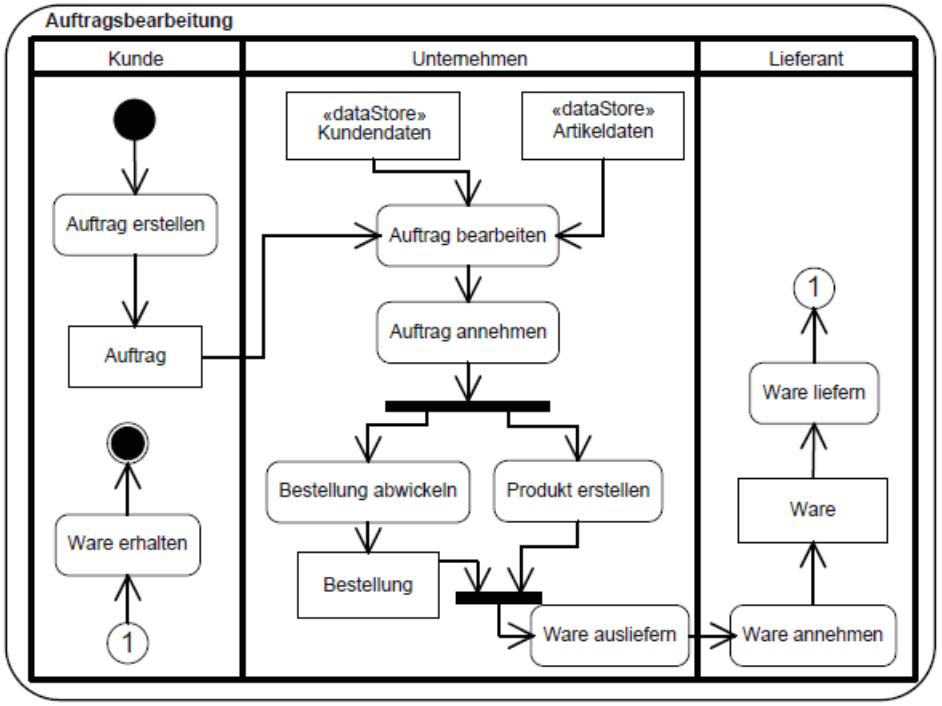
- Alle Benutzer werden im Puffer "Benutzer" gespeichert (Datenbank).
- Ausgewählte Benutzer werden in einem transienten Puffer zwischengespeichert und erst für die Versendung von Einladungen wieder entnommen.

Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Grundlagen (6/7)
Partitionen (swim lines)

Partitionen / **Verantwortlichkeitsbereiche** (*swim lanes*) erlauben die Gruppierung von Knoten und Kanten einer Aktivität nach bestimmten Kriterien

- Zuordnung der Aktivitäten zu anderen Strukturen z.B. zu entsprechenden Klassen, Use Cases, oder Organisationseinheiten
- Logische Sicht auf eine Aktivität zur Erhöhung der Übersichtlichkeit und Semantik des Modells

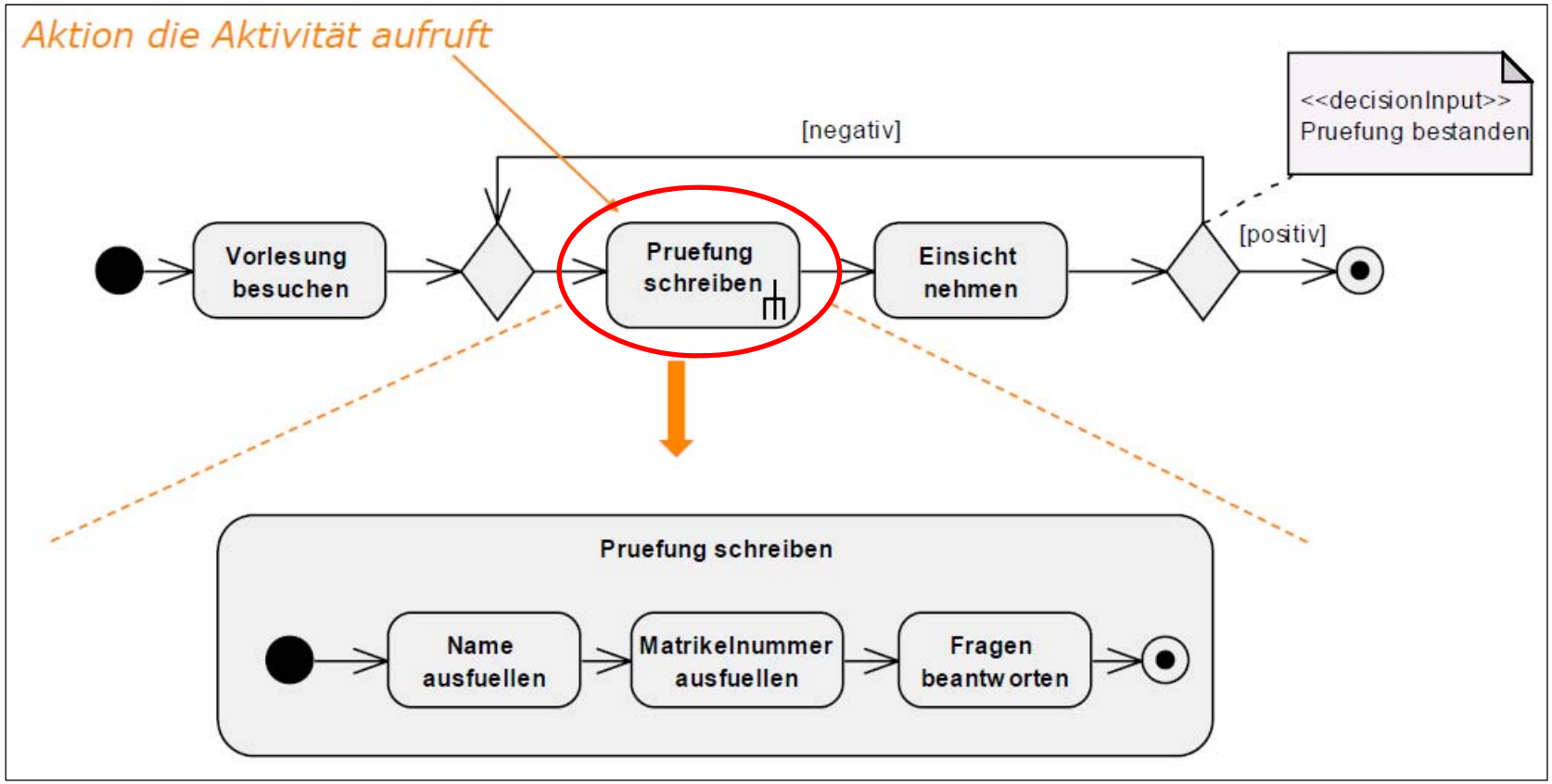


Vereinfachte Darstellung einer Auftragsbearbeitung

Quelle: Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012, S.48

Aktivitätsdiagramm: Grundlagen (7/7)

Schachtelung von Aktivitäten



- Aktivitäten können wiederum Aktivitäten aufrufen
- So können Details in eine tiefere Ebene ausgelagert werden

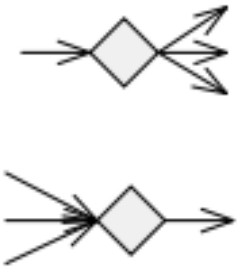
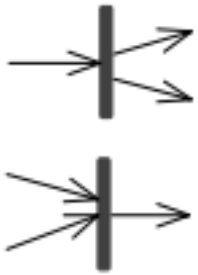
Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Grundelemente (1/5)

Name	Syntax	Beschreibung
Aktionsknoten		Repräsentation von Aktionen (Aktionen sind atomar!)
Initialknoten		Kennzeichnung des Beginns eines Ablaufs einer Aktivität
Aktivitätsend-knoten		Kennzeichnung des Endes ALLER Abläufe einer Aktivität
Ablaufend-knoten		Kennzeichnung des Endes EINES Ablaufes einer Aktivität

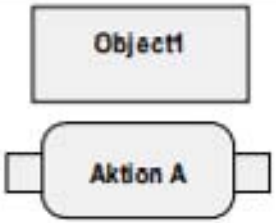
Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

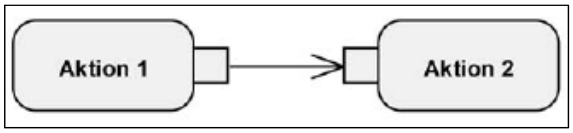
Aktivitätsdiagramm: Grundelemente (2/5)

Name	Syntax	Beschreibung
Entscheidungs-/ Vereinigungs-knoten		Aufspaltung/Zusammenführung von alternativen Abläufen
Parallelisierungs-/ Synchronisations-knoten		Aufspaltung eines Ablaufs in nebenläufige Abläufe / Zusammen-führung von nebenläufigen Abläufen in einen Ablauf

Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Grundelemente (3/5)

Name	Syntax	Beschreibung
Transition		Verbindung der Knoten einer Aktivität
Aktivitätsaufruf		Schachtelung von Aktivitäten
Objektknoten, Pins		Beinhalten Daten und Objekte



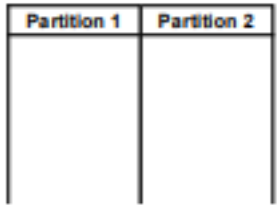
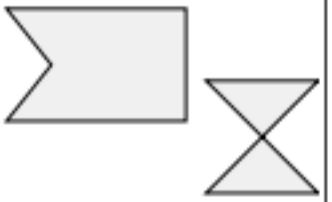
Ausgabe- und Eingabepins



Äquivalente Darstellung mit Objektknoten

Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Grundelemente (4/5)

Name	Syntax	Beschreibung
Partition		Gruppierung von Knoten und Kanten innerhalb einer Aktivität
Signal		Übermittlung eines Signals an einen Empfänger
asynchrones Ereignis/ Zeitereignis		Warten auf ein Ereignis bzw. einen Zeitpunkt

Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Grundelemente (5/5)

Name	Syntax	Beschreibung
Exception Handler		Aktivität wird bei Ausnahme ausgeführt
Unterbrechungs bereich		Wird der Unterbrechungsbereich über die Unterbrechungskante verlassen, so werden alle in der Region vorhandenen Token gelöscht

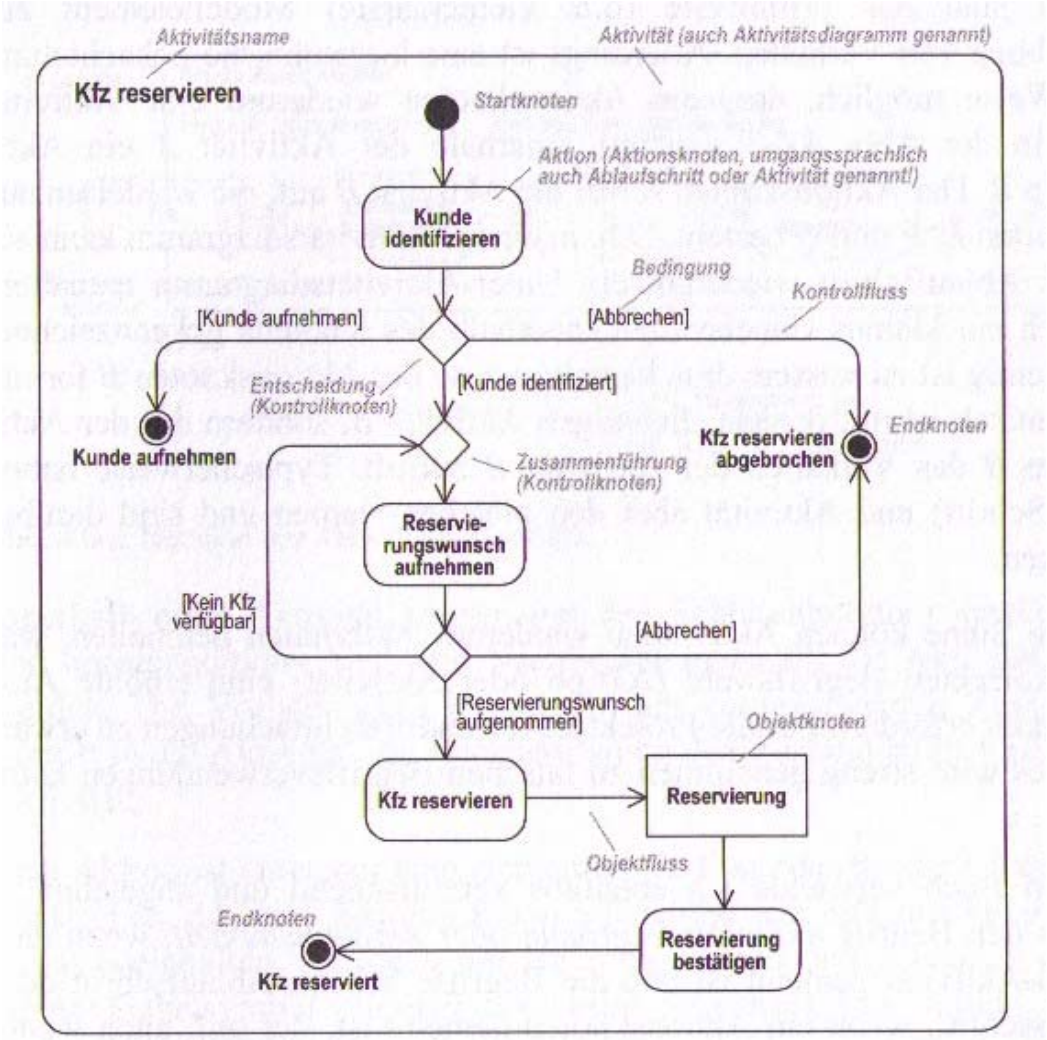
BEACHTEN:

- Signale und Ereignisse sowie
- Exception Handler und Unterbrechungsbereich

werden im Rahmen dieser Vorlesung nicht weiter betrachtet und gehören NICHT zum Prüfungsumfang!

Quelle: M. Seidl et al.: UML @ Classroom: Eine Einführung in die objektorientierte Modellierung, dpunkt.verlag 2012

Aktivitätsdiagramm: Abschließendes Beispiel



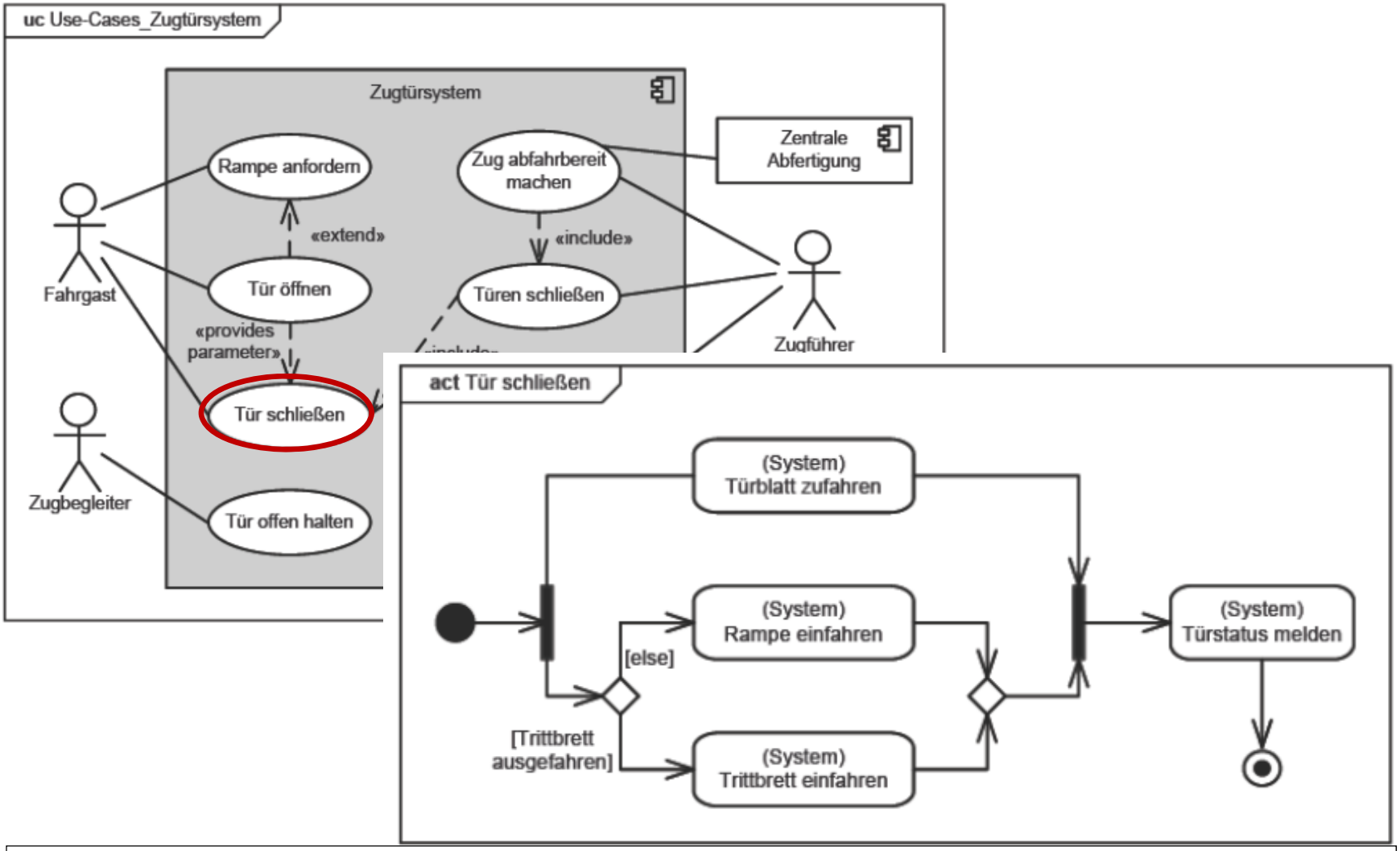
Quelle: Oestereich, B.: Objektorientierte Softwareentwicklung – UML 2.0, Oldenbourg-Verlag, 2004

Aktivitätsdiagramm: Einsatzbereiche

- GP-Modellierung
  - Modellierung von Arbeitsabläufen
  - Darstellung der zugeordneten Organisationsstrukturen
- Analysephase
  - Untersetzung fachlicher Zusammenhänge und Abhängigkeiten die sich hinter einem Anwendungsfall verbergen
  - Beschreibung von Sachverhalten aus mehreren Anwendungsfällen bzw.
  - Darstellung anwendungsfallübergreifender Aspekte
- Design
  - Detaillierte Spezifikation mit Implementierungsbezug
  - Detaillierte Darstellung des Ablaufverhalten von Klassen und deren enthaltenen Operationen
- Aktivitäten oder Aktivitätsdiagramme sind i.d.R.
  - einem Anwendungsfall,
  - einer Klasse,
  - oder einer Operation

  zugeordnet.

Aktivitätsdiagramm: Untersetzung von Use Cases



Quelle: Rupp, Ch.: UML 2 glasklar.Praxiswissen für die UML-Modellierung, Hanser Verlag, 2012, S.52

Übung 2-5: Aktivitätsdiagramm
Aufgabenstellung

Muss

a) Untersetzen Sie das von Ihnen erstellte Anwendungsfalldiagramm unter Verwendung eines oder mehrerer Aktivitätsdiagramme. Die Verwendung von Anwendungsfalldiagrammen sollte sich z.B. auf Geschäftsregeln, fachliche Abhängigkeiten oder auch Gültigkeitsaspekte beziehen.

- Festlegung der aus fachlicher Sicht zu modellierenden Abläufe
- Dokumentieren Sie ihr Projektergebnis zur weiteren Verwendung

Muss

b) Mindestens 2 Gruppen präsentieren ihre Ergebnisse